Artificial Intelligence and Robotics

A Complete Course in Plain English

ARTIFICIAL INTELLIGENCE AND ROBOTICS

Volume One

Introduction to Artificial Intelligence

TBUGE MECHATRONIX

Artificial Intelligence and Robotics, Volume One: Introduction to Artificial Intelligence

First edition. TBUGE Mechatronix.

This volume stands alone. It assumes no previous study of computing, no mathematics beyond secondary school, and no equipment beyond a pen, some paper and a calculator. The volumes that follow take up the ideas introduced here and build a working knowledge of robotics, machine learning, control, perception, planning and the ethics that bind them.

The series runs to forty-eight volumes, one for each course in the programme. They are designed to be read in order, but each is written so that a reader who comes to it late can still follow the argument.

Course codes have been left out on purpose. A subject should be named, not numbered.

Contents

Preface **6**

How This Volume Is Arranged **8**

Part One · The Question of Intelligence **10**

The Moving Target 11

Two Winters, Three Ideas 17

The Anatomy of an Agent 23

Part Two · Deciding **28**

Search: The First Real Trick 29

What the Experts Knew 35

Evidence, Not Fact 41

Part Three · Learning and Measuring **46**

Learning from Examples 47

The Cost of Being Wrong 52

How Wrong Is It, Really? 57

Part Four · The Senses **62**

Pictures, Sounds, Words 63

Part Five · The World **68**

The Chain Around the Model 69

What Machines Still Cannot Do 74

Where This Leaves You 79

Glossary **82**

Sources and Further Reading **85**

Preface

Somewhere on your phone there is a chess program that plays far better than any human being who has ever lived. It was not there when Deep Blue beat Garry Kasparov in 1997, in a room in New York, in front of cameras. Back then, a machine that could win at chess was front-page news for a fortnight. Today it is an app that comes free with an operating system, and nobody calls it intelligent.

The same thing has happened to reading handwriting, recognising speech, finding a route across a city and translating a sentence. Each of these was a research triumph that took decades. Each is now ordinary. We do not stop using them; we stop calling them artificial intelligence. The name moves on to whatever is still hard.

That habit is worth understanding before you learn anything else about this field, because it shapes the way the subject is discussed in newspapers, in boardrooms and in government. A book about artificial intelligence that does not begin by asking what the words mean will spend the rest of its length chasing a target that is already moving. So Volume One begins there.

This volume is the first course of a four-year programme. It is the one course with no prerequisites, and it is the one that everything else leans on. Its job is to give you the vocabulary, the habits of thought and the small set of ideas that the other forty-seven volumes assume. You will not train a deep network here. You will not build a robot. You will learn what an intelligent system is made of, how it searches, how it reasons with rules, how it learns from examples, how it handles uncertainty, how it begins to see and read and listen, and how the whole thing so often fails in the world after it has succeeded in the laboratory.

Why the language is plain

There is a fashion in technical writing for difficulty. It is supposed to signal rigour. In practice it often signals that the writer has not yet decided what they mean, because a writer who knows what they mean can usually say it simply.

So this book uses short sentences. It defines every technical word the first time it appears, and then uses it freely. It gives numbers with units. It prefers one worked example to three pages of general statement. When it uses mathematics, it says in words what the mathematics does before it shows the symbols, because a formula without a sentence is a puzzle, not an explanation.

This is not because the material is easy. It is not easy. Learning how a robot estimates where it is while it draws a map, or why a neural network fails on a picture it has never seen, is genuinely hard work. Plain language does not remove the difficulty. It removes the extra difficulty that badly written English adds on top.

What this book will not do

It will not tell you that machines are about to become conscious, and it will not tell you that they will never amount to much. Both claims are made constantly and both are usually made without evidence. What the evidence supports is narrower and more interesting: these systems are extraordinarily good at finding patterns in large collections of examples, they fail in ways that are specific and predictable, and the consequences of those failures depend on where the system is deployed and who bears the cost.

It will not pretend that the ethical questions can wait until the engineering is finished. They cannot. Every design decision about a sensor, a threshold, a dataset or an interface is also a decision about who is served, who is watched, who is inconvenienced and who is at risk. The final chapters of this volume begin that conversation, and three later volumes continue it.

And it will not encourage you to skip the arithmetic. The arithmetic in this volume is small: percentages, averages, one probability rule, one cost comparison. It is not there to make the book look serious. It is there because these few calculations dissolve most of the confusion in the field, and because a person who can compute an expected cost is very hard to mislead.

Who this is for

It is for a student beginning a degree or diploma in artificial intelligence, robotics, mechatronics or computer engineering, in a classroom with a good teacher or alone with a notebook. It is for a teacher who needs an order in which to explain things. It is for an engineer from another field who has to talk to a robotics team without pretending. It is for anyone who has read a dozen articles about artificial intelligence and come away with the suspicion that something is being left out.

One more thing. The exercises in this volume require almost nothing: paper, a pen, a phone, patience. That is deliberate. Equipment should never be the reason a student cannot begin, and there is no idea in this volume that cannot be met with a pencil.

How This Volume Is Arranged

The volume is in five parts, and the parts are stages of an argument rather than a list of topics.

Part One asks what intelligence means when we say a machine has it, where the idea came from, and what an intelligent system is actually made of. By the end of it you will be able to draw any such system as a loop of sensing, deciding and acting, and say why the shape of its world changes the design.

Part Two is about deciding. Machines that play games, route deliveries and diagnose faults do not learn from data in the first instance; they search, or they follow rules, or they weigh evidence. These are the oldest and most reliable tools in the field, and they are still inside almost everything built today.

Part Three is about learning, and about measurement. A model that has learned from examples is only as good as the honesty of its evaluation, so half of this part is devoted to the question that most introductions skip: how do you know whether the thing works?

Part Four is a short visit to the senses. Vision, hearing and language are three very different problems that turn out, at the level of method, to be the same problem three times over.

Part Five follows the system out of the laboratory. It is about the chain around the model, the failures that happen after the success, and the limits and

risks that a person building these systems has to hold in mind.

Each chapter ends with work to do: sometimes a set of problems, sometimes an experiment on paper, sometimes a question to argue about with someone else. The exercises are not decoration. Thinking you understand something and finding that you cannot compute it are two different states of mind, and the second is the one that teaches.

A note on the numbers. Every figure in this volume is either derived in the text or drawn from a published source named in the text. Where a figure is an example invented to make a point clear, it says so. Where a number is uncertain, it says so as well. You should get into the habit of asking for that distinction, because you will meet a great many confident claims that do not offer it.

PART ONE

The Question of Intelligence

CHAPTER ONE

The Moving Target

On 11 May 1997, in the Equitable Center in Manhattan, a machine called Deep Blue beat Garry Kasparov in the sixth and final game of a match. Kasparov was the reigning world chess champion. Deep Blue was a pair of refrigerator-sized cabinets built by IBM. The match had drawn worldwide attention for two weeks; the prize was more than a million dollars; the front pages treated the result as a turning point in the relationship between people and their tools.

Ten years later, a chess program that could beat almost any human grandmaster ran on a laptop, and grandmasters used it to prepare rather than fearing it. Twenty years later it ran on a phone, silently, inside an app that also told you what time the tide came in. Nothing about the program had become less capable. Quite the opposite: the phone version was stronger than Deep Blue. It had simply stopped being remarkable, and once a thing stops being remarkable, we stop calling it intelligent.

This is the first fact about artificial intelligence, and the one that causes the most confusion. The word does not describe a fixed property that a machine either has or lacks. It describes a boundary that keeps moving. Whatever the field manages to build gets reclassified as ordinary engineering, and the name moves on to whatever is still unsolved. Optical character recognition was a triumph in the 1980s and is now a camera feature. Route finding was a landmark of the 1960s and is now a blue line on a map. Speech recognition took fifty years and is now a button.

If you take nothing else from this chapter, take this: the label tells you almost nothing. If you want to know what a system can do, you have to look at the system.

Judging by What a Thing Does

Philosophers have argued for centuries about what intelligence is, and the argument is not over. This book takes a different route, on purpose. It treats intelligence as a way of behaving, not as an inner substance.

The definition we will use is plain: **intelligence is the ability to choose actions that reach a goal in situations you have not seen before**. Three pieces of that sentence matter. There is a goal, so there is a difference between success and failure. There is a choice, so the thing is not merely following a script. And there is novelty, because a system that works only on the exact situations it was built for has demonstrated memory, not intelligence.

Test it on three machines. A calculator is fast and never chooses; it does exactly what it is told, and it would do the same thing in any situation. A chess engine chooses, but only among moves on a board, and its world is perfectly known; it is intelligent inside a very small box. A self-driving car chooses among steering and braking actions, in a world that is partly hidden, moving and full of people; it is doing something much more like what a person does, and it is much worse at it than a person is.

The definition also gives you a way to ask useful questions. For any system, ask: what can it sense? What actions can it choose from? What counts as the goal? And how many situations outside its training ground can it handle? Four questions, and you can answer them for a spam filter, a factory arm, a hospital triage tool or a chat assistant.

These are deliberately the same four questions an engineer asks when writing a design specification. That is not an accident. In this book, intelligence is not a mystery to be admired. It is a property of a system, and properties of systems can be specified, built, measured and improved.

What the Word Hides

There are two temptations when reading about artificial intelligence, and both come from taking the name too seriously.

The first is to imagine that a system which is impressive in one place will be impressive in another. It will not. The chess engine that beat the world champion could not tell a chair from a table. It could not recognise a chessboard in a photograph, or explain its moves to a beginner, or play any other game without

being rebuilt. The technical word for this is narrowness. Almost every system you will meet in this book is narrow: excellent inside its task, helpless outside it.

The second temptation is the opposite one: to hear that a system is narrow and conclude that it is trivial. Narrow systems run factories, read postal addresses, screen loans, recommend what you watch next and, increasingly, drive. Narrowness is not the same as weakness. A scalpel is narrow too.

What narrowness does mean is that claims must be tied to tasks. "This system understands language" is not a claim you can test. "This system answers questions about a set of technical manuals with 82% accuracy, and invents an answer rather than saying it does not know in 6% of cases" is a claim you can test, argue with, and decide whether to deploy. The whole discipline of evaluation, which Part Three of this volume takes up, exists to turn the first kind of sentence into the second.

A Number Without a Baseline

Here is a small piece of arithmetic that will save you from more bad claims than any other single idea in this book.

Suppose you build a filter that decides whether an email is spam. You test it on a thousand messages and it is right 900 times. Ninety per cent. Is that good?

You cannot know, because you do not know what a stupid method would score. Imagine that only 100 of those thousand messages are spam, and 900 are legitimate. A filter that never checks anything, and simply says "not spam" to every message that arrives, is right 900 times out of a thousand. It has the same score as yours, and it contains no intelligence whatsoever. Its author could publish the same headline.

This is why every experiment in this book begins with a **baseline**: the score that a deliberately simple method achieves. The baseline is the bar that intelligence has to clear. In the spam case, the bar is already at 90%, and a filter that merely matches it has earned nothing.

Now look more closely at your filter. It catches 60 of the 100 spam messages and wrongly blocks 30 real ones. Among spam, it is right 60% of the time. Among ordinary mail, it is right 870 / 900, or 96.7%, of the time. Those numbers describe a product people might tolerate for a week before switching filters, because missing forty out of a hundred junk messages and hiding thirty

real ones is a bad day at the office.

The single figure of 90% concealed all of it. That is the lesson: a score without a baseline and without a breakdown of its mistakes is not a result. It is a slogan wearing the clothes of a result.

The Wolf That Was Really Snow

In 2016, a group of researchers at the University of Washington asked a question that more people should ask about machine learning systems: when this model says “wolf”, what is it looking at?

The model in question was a classifier trained to tell photographs of huskies from photographs of wolves, and it appeared to do so very well. The researchers analysed which parts of each image pushed the decision one way or the other. What they found was a small scandal. The model was not looking at the animal. It was looking at snow.

Almost every wolf photograph in the dataset had been taken outdoors, in winter, on snow. Most husky photographs had been taken in homes, parks or summer fields, without snow. So “there is snow on the ground” predicted “wolf” almost perfectly, and the model had learned the shortcut rather than the animal.

On the data it was given, the model was accurate. On a wolf photographed in a desert, or a husky standing in snow, it would have failed, and nobody would have known until it did.

Notice how neatly this fits the definition at the start of the chapter. The system did not choose actions to reach a goal in new situations. It found one correlation that held in one collection of pictures and would have broken the moment the situation changed. That is the difference between a system that scores well and a system that is intelligent in any sense worth the word, and the difference is invisible in a single accuracy number.

The practical habit is easy to state and hard to keep: whenever a model does surprisingly well, ask what it could be using that you did not intend. Then build a test in which that shortcut is unavailable. If the score collapses, you have found the shortcut, and you have learned more than a hundred extra training examples would have taught you.

What the Definition Costs

Treating intelligence as behaviour has a cost, and it is only fair to name it.

A system can behave intelligently, in the sense used here, without any inner life at all, and a system can have inner states that its behaviour does not reveal. Philosophers have made careers of this gap. John Searle's Chinese room argument, published in 1980, asks you to imagine a person in a room who follows rules for manipulating Chinese characters, producing replies that look fluent to anyone outside, without understanding a word of Chinese. If the room passes the test, has it understood? Searle said no. Others said the room as a whole understands. The argument is not settled, and this book does not settle it.

What the book does is take a practical position. For the purposes of building, testing and regulating machines, behaviour is what we can observe, measure and be responsible for. If a system's outputs can harm someone, the harm is real whether or not anything is happening inside. That is why the definition in this chapter is behavioural: not because the philosophical question is unimportant, but because it is not the question an engineer can answer at the bench.

You will meet this tension again in the volumes on ethics, where the question is not whether a machine understands, but whether the people who built and deployed it can give an account of what it did.

Problems

1. Write down five systems described as “AI” that you used this week. For each, fill in three columns: what it senses, what it chooses, and what would count as failure. Mark each one narrow or general.

2. For the system you know best, invent a test. Write the task, the measure of success, and a baseline that uses no learning at all. Predict the baseline's score before you predict the clever system's score. If your clever system does not clearly beat the baseline, the test is too easy to be interesting.

3. Here is a table from a small experiment. A classifier was tested on 200 photographs, 40 of which showed a cracked weld. It flagged 50 photographs as cracked. Of those 50, 30 really were.

	Flagged	Not flagged
Actually cracked (40)	30	10
Actually sound (160)	20	140

Compute the accuracy, then compute the two numbers that matter: of the flagged photographs, what fraction were cracked, and of the cracked welds, what fraction were found. Write two sentences explaining why a factory manager might reject this system even though it is accurate most of the time.

4. Find a news story that describes a system as intelligent. Rewrite its central claim in the form “on this task, with this measure, it scores X, compared with a baseline of Y”. If you cannot fill in the blanks, say so, and say what you would need to ask the author.

CHAPTER TWO

Two Winters, Three Ideas

In the summer of 1956, a group of about ten men spent several weeks at Dartmouth College in New Hampshire, talking about whether machines could be made to think. The proposal that brought them there, written the year before by John McCarthy, Marvin Minsky, Nathaniel Rochester and Claude Shannon, contained a sentence that has been quoted ever since: that every aspect of learning or any other feature of intelligence can in principle be so precisely described that a machine can be made to simulate it.

It is worth pausing on how confident that sentence is. Not “some aspects”. Not “simple features”. Every aspect, in principle, and the authors expected a generation of work to settle it.

Seventy years later, the work is still going, and the argument about how far it has got is louder than ever. But the shape of that seventy years is not a straight line of progress. It is a loop, repeated three times, and knowing the loop is the best defence against both the hype and the despair that the subject produces in equal measure.

The Paper That Started It

The technical foundation was laid in 1936, when Alan Turing, then twenty-four, published a paper describing an abstract machine that could compute anything computable by following a finite list of simple steps. The point was not to build such a machine. The point was to give a precise meaning to the words “a procedure that can be carried out mechanically”. Once you have that meaning, the question “can a machine think?” stops being poetry and becomes something you can argue about with evidence.

Fourteen years later, Turing wrote the paper that gave the field its founding experiment. Rather than defining thinking — a definition nobody could agree on — he replaced the question with a test. Put a person, a machine and a judge

in separate rooms. Let the judge type questions to both. If the judge cannot reliably tell which is which, said Turing, we have no good reason to deny that the machine thinks.

The move is characteristic of the field at its best: when a question is too vague to answer, replace it with an experiment. It is also characteristic of the field at its worst, because a machine can pass the test by being good at conversation without understanding anything, which is more or less what happened seventy years later, and which is why the test is now a footnote rather than a goal.

Machines That Followed Rules

The first generation believed that intelligence was the manipulation of symbols. If a doctor's knowledge could be written down as rules, a program could follow the rules and give advice. This produced genuine, useful systems.

MYCIN, built at Stanford in the 1970s, advised doctors on bacterial infections of the blood. It asked questions, weighed evidence, and recommended an antibiotic. In blind tests it performed about as well as human specialists. It had roughly six hundred rules, each argued over by clinicians, and it could explain its reasoning by listing the rules it had used. It was never used routinely in hospitals, for reasons that had nothing to do with accuracy: doctors did not want to type answers to long lists of questions while a patient waited, the system ran on specialised hardware, and nobody knew who would be legally responsible if the advice was wrong.

XCON, built at Carnegie Mellon for the Digital Equipment Corporation, configured orders for VAX computers. A customer order might specify dozens of components that had to fit together physically, electrically and in software; human specialists took hours and sometimes made expensive mistakes. XCON took minutes, and by the company's own estimate saved tens of millions of dollars a year. For a while it was the flagship of an entire industry. Then each new product line needed new rules, each rule interacted with rules written years earlier by engineers who had moved on, and the knowledge base grew into the thousands. Maintaining it became a specialist job in itself. The system never became wrong. It became too expensive to keep right, which in business is the same thing.

This pattern — real success, then a wall made of maintenance, cost and common sense — is the pattern of the first wave, and it is the reason the field's history is a loop rather than a line.

The First Winter

The first collapse came earlier than most people realise. In 1969, Marvin Minsky and Seymour Papert published a careful mathematical analysis of the perceptron, a simple learning machine built by Frank Rosenblatt a decade earlier. Their book showed that a single-layer perceptron could not compute even the logical function “exclusive or”, a case where the answer is yes if either input is yes but not both. The result was narrow. It was read, unfairly but widely, as proof that learning machines were a dead end. Funding for neural network research collapsed.

In 1973, the British government commissioned a review of the field from Sir James Lighthill, a distinguished applied mathematician. His report was sceptical, and British funding was cut. In the United States, defence funding for open-ended AI research was reduced around the same period. The years from roughly 1974 to 1980 are usually called the first AI winter: not a period in which the ideas died, but one in which the money did.

The second winter came in the late 1980s, and again the cause was economic. Expert systems were running on special-purpose computers, and those machines were expensive. When ordinary desktop workstations became fast enough to run the same software, the hardware business collapsed almost overnight. Around 1987, the market for Lisp machines disappeared. The companies that had bet on it disappeared with it, and with them a great deal of the credibility of the expert-systems wave.

Two winters taught the field a lesson it has never quite forgotten: a method can be correct, elegant and still too expensive to survive. That lesson is why every chapter in this volume asks not only whether a method works, but what it costs to keep working.

The Learning Turn

While the rule-based wave was freezing, a different idea was warming up. In 1986, a paper by David Rumelhart, Geoffrey Hinton and Ronald Williams in *Nature* popularised a practical method for training networks with more than one layer, called backpropagation. The technique had been discovered and rediscovered several times before, but 1986 was the moment it became usable and widely known. It removed the objection of 1969 at a stroke: networks could now learn things that a single layer could not represent.

The 1990s took a further step away from hand-written rules, towards statistics. Instead of asking an expert what the rules were, you collected data and estimated probabilities. This was less glamorous, far more reliable, and it worked in speech recognition, handwriting, credit scoring and machine vision. The field changed its vocabulary: fewer “knowledge bases”, more “training sets” and “error rates”.

In May 1997, in the middle of this turn, Deep Blue beat Kasparov. It is worth being precise about how: Deep Blue used special-purpose hardware to examine around two hundred million positions per second, plus an evaluation function tuned with the help of strong human players. It did not learn from data. It was the last great triumph of the search-and-rules tradition, arriving just as the statistical tradition was taking over.

Scale Arrives

In 2009, researchers at Princeton and Stanford released ImageNet, a dataset that eventually contained more than fourteen million labelled photographs. The following years produced a competition to classify a thousand categories of object from images. In 2012, a deep network called AlexNet won that competition with an error rate of about 15%, against roughly 26% for the second-placed entry. It was trained on graphics processors built for video games.

The pattern was set: bigger networks, more data, more computing power. In 2016, AlphaGo beat Lee Sedol at Go, a game whose branching factor is so large that searching it was long considered hopeless, and it did so partly by learning an evaluation function from millions of games and then from games against itself. In 2017, a paper called “Attention Is All You Need” introduced the transformer, an architecture that processes sequences in parallel and handles

context with a mechanism called attention. In November 2022, chat models built on transformers reached the public, and the argument about artificial intelligence moved from the technology pages to the family dinner table.

Notice what did not happen at any point in this story. The old ideas did not die. Chess engines still search. Rule systems still run factories, aircraft and payment systems. Learning sits on top of both. The history is not a series of replacements; it is a series of additions, each one surviving because it solved a problem the others could not.

Period	Dominant idea	Characteristic success	Characteristic failure
1956–1975	Symbols and search	Puzzles, games, simple proofs	Real-world messiness
1975–1987	Expert rules	Medicine, configuration, planning	Cost, brittleness, common sense
1987–2012	Statistical learning	Speech, handwriting, recommendation	Needs labelled data
2012–today	Deep learning at scale	Vision, language, games	Data hunger, opacity, expense

What the Loop Teaches

Three lessons follow from this history, and they are the reason it earns a chapter instead of a paragraph.

The first is that each wave was a response to the wall of the previous one. Rules failed because knowledge had to be written by hand; learning answered that. Learning failed because it needed features designed by hand; deep learning answered that. Deep learning is now meeting its own walls: data, energy, reliability and trust. The next wave will be an answer to one of those, and you can predict its shape by asking which wall it removes.

The second is that the winters were about money and maintenance, not about ideas being wrong. This matters to you personally. Most projects do not fail because the algorithm is false. They fail because the data cannot be collected, the system cannot be maintained, the integration takes longer than the research, or the organisation loses interest. Keep that in view when you plan your own work.

The third is that every wave renames itself. What was called cybernetics, then symbolic AI, then expert systems, then machine learning, then deep learning, then generative AI, is one continuous attempt to build machines that choose well. The names are useful for dating a conversation. They are not useful for deciding what is true.

Exercises

1. Draw a horizontal line and mark it 1940, 1960, 1980, 2000, 2020. Place these on it: Turing's paper on computable numbers (1936), the Dartmouth workshop (1956), the perceptron (1958), the perceptron critique (1969), the Lighthill report (1973), MYCIN, XCON, the Lisp machine collapse (1987), backpropagation (1986), Deep Blue (1997), ImageNet (2009), AlexNet (2012), AlphaGo (2016), the transformer paper (2017), public chat models (2022). Above the line, write the dominant idea at each point. Below it, write the wall that idea ran into.

2. MYCIN matched human specialists and was never used. XCON saved millions and became unmaintainable. Write one paragraph on what these two outcomes have in common, and one on what they do not.

3. Find a news article about artificial intelligence from five years ago. Identify the wall that its predictions did not see coming. Then find one from this month and predict the wall it is not seeing. You will be wrong, but the exercise is the point.

CHAPTER THREE

The Anatomy of an Agent

A thermostat is a poor thing to call intelligent, and yet almost everything that deserves the name is built the same way. Something measures the world. Something decides what to do about it. Something changes the world. The measurement comes back, and the cycle repeats. The thermostat does this with a single temperature and a switch. A self-driving car does it with a dozen sensors and a steering wheel. The difference is in the size of the loop, not in its shape.

This chapter gives that shape a name and a set of boxes, because a surprising number of confused arguments about artificial intelligence are really arguments about which box is being discussed. When someone says a robot “decided” to do something, ask what it sensed, what it could have chosen instead, and how it knew the outcome. Usually one of the three is missing, and the sentence collapses.

A Box on the Wall

Take the thermostat seriously for a minute, because it is the smallest complete example of the thing this book is about.

It has a temperature sensor, a switch that turns the heating on or off, and a target. The naive policy is to heat whenever the room is below the target and stop when it reaches it. That policy works in theory and fails in practice, because real rooms have thermal mass and real sensors have noise: the heating clicks on and off every few seconds, and the relay that does the switching wears out within a season.

So thermostats use a band. Heat until the room reaches 21.5 °C. Then do nothing until it falls to 20.5 °C, and heat again. Inside the band, the heater keeps whatever state it had. Written as a table, the policy looks like this:

Temperature	Heater
Below 20.5 °C	On
20.5 °C to 21.5 °C	Keep previous state
Above 21.5 °C	Off

Three lines, one number sensed, one actuator, no memory beyond the previous state. That is an agent. It is not intelligent in any interesting sense, but it is complete: it senses, it decides, it acts, and it has a goal it can miss. Once you can see the thermostat as an agent, you can see a factory robot or a chat assistant the same way, and the questions you ask about them become much sharper.

Four Boxes and a Loop

The standard description of an agent has four parts, and the acronym for them is PEAS: **P**erformance measure, **E**nvironment, **A**ctuators, **S**ensors. It is worth the ninety seconds it takes to memorise, because it is the first page of every serious design document you will ever write.

Take a delivery robot in a hospital. It carries blood samples between wards, uses lifts, and shares corridors with staff, patients and trolleys.

Performance. Deliveries per hour. Zero collisions with people. Zero lost or mislabelled samples. Battery never below 20% at the end of a shift. Those four are not the same kind of number: one is a rate to maximise, two are constraints that must never be violated, and one is a condition for the machine to finish its day. Writing them as a flat list and pretending they are interchangeable is the most common error in the whole design.

Environment. Corridors with polished floors, lifts with closing doors, people who move unpredictably, trolleys left in the way, and the occasional spill. The robot's world is not a diagram. It is a building with a cleaning schedule.

Actuators. Two drive wheels, a lift-call button, a screen, a speaker.

Sensors. A laser scanner, cameras, bumpers, wheel encoders, a battery monitor, and a downward sensor that notices when the floor has stopped being a floor.

Now something useful happens. The PEAS description tells you what the hard part will be before you write a line of code. The environment has people in

it, which means the world changes while the robot is deciding. The performance measure contains constraints that must never be violated, which means the decision procedure needs a safety layer that does not depend on the main decision procedure. The actuators include a speaker, which means the robot is expected to explain itself to humans, which is a design commitment, not a feature.

Worlds Have Shapes

Two agents with the same sensors can need completely different algorithms, because their environments differ in ways that decide what is possible.

Fully or partially observable. In chess you can see every piece. In a corridor you cannot see through a wall. Partial observability forces memory and inference: the agent must keep a picture of what it has seen and reason about what it cannot see.

Deterministic or stochastic. A chess move has a known result. A wheel on a wet floor may slip. Stochastic environments push you from rules towards probability, which is the subject of the sixth chapter.

Static or dynamic. In a puzzle, the world waits while you think. In traffic, it does not. Dynamism puts a deadline on every decision, and a deadline changes the kind of algorithm you can use.

Discrete or continuous. A grid has finitely many squares. A real corridor has infinitely many positions. Continuous spaces need different mathematics, which is why the volumes on kinematics and planning exist.

Single-agent or multi-agent. A robot alone in a warehouse chooses freely. A robot among people must predict what the people will do, knowing that they are predicting it too.

Here is the practical consequence, and it is worth writing on a card. A fully observable, deterministic, static game can be solved by searching ahead. A partially observable, stochastic, dynamic street cannot. The first is a puzzle. The second is robotics. A great many student projects fail because a puzzle method has been aimed at a street.

The Ladder

Agent designs can be arranged from simple to complex, and each rung adds a capability at a cost.

At the bottom is the **reflex agent**: if this, then that. The thermostat. Fast, predictable, and helpless the moment the condition does not contain enough information to decide.

Above it is the **model-based agent**, which keeps an internal picture of the world. A robot that remembers a chair was there two seconds ago can still avoid it when a person walks between the robot and the chair. The cost is memory, and the new risk is a stale model: the robot confidently avoiding a chair that was moved.

Above that is the **goal-based agent**, which chooses actions by planning a route to a goal instead of following a fixed rule. This is where search enters, and it is the subject of the next chapter.

Above that is the **utility-based agent**, which weighs competing goals and picks the best trade-off. A delivery robot that balances speed against battery life and passenger comfort is doing this. The hard part is not the algorithm; it is deciding what the numbers should be.

At the top sits the **learning agent**, which improves its own policy from experience. A vacuum robot that adapts to a new flat, or a language model that is trained on more text, belongs here.

The cheapest illustration of the ladder is the domestic vacuum robot. The first mass-market model, launched in 2002, was a reflex agent: it drove in a straight line until it bumped something, turned, and carried on. It had no map, no memory and no plan. It also sold in enormous numbers, because it met the measure that mattered to buyers — the floor was cleaner when it finished, at a price people would pay. Later models added memory of obstacles, then systematic coverage patterns, then mapping with a camera or a laser, then scheduling that cleaned the rooms that got dirty most often. Each rung cost more to build, more to test and more to explain, and each answered a complaint from a real user.

That is the correct order of operations, and it is the opposite of the order that excites engineers. Start at the lowest rung that satisfies the measure. Climb when a real person has a real complaint. A reflex agent that meets the requirement is a better engineering answer than a learning agent that nearly does.

The Measure Bites Back

There is a trap at the centre of all of this, and it deserves its own section because it has ruined more systems than any algorithm.

The agent will optimise exactly what you measure, including the parts you did not mean. Reward a cleaning robot for distance travelled and it will drive in circles all day, because circles are excellent for distance. Reward a delivery robot for speed and it will eventually hit someone, because people are slow. Reward a support chatbot for the length of its answers and it will learn to be tedious.

Economists call the general phenomenon Goodhart's law: when a measure becomes a target, it ceases to be a good measure. The engineering version is blunter. The performance measure is the steering wheel of the project, and if you attach it to the wrong shaft, the vehicle goes where the shaft points.

The defence is not to write a better number. It is to write the measure as two things: something to maximise, and something that must never happen. “Deliver more samples per hour, and never make contact with a person” is harder to game than “maximise efficiency”, because the second clause cannot be traded away for the first.

Things to Try

1. Write a PEAS description for a lift. It senses calls from floors, carries people, and has a goal. Then classify its environment on the five properties. Which one makes the lift's problem easier than it first appears?

2. Classify these three environments, then name the method each rules out: a chess position, a hospital corridor, a warehouse with twenty robots and one human supervisor.

3. The thermostat policy above has a gap. What happens if the temperature sensor fails and reports nothing at all? Write the missing line of the table, and decide whether “do nothing” is safer than “heat”.

4. Think of a system you use that was clearly designed around the wrong measure. Describe the measure it appears to optimise, the measure its users care about, and what a better pair of clauses would look like.

PART TWO

Deciding

CHAPTER FOUR

Search: The First Real Trick

At two in the morning, in a distribution centre outside Rotterdam, a robot the size of a dining table receives an order: collect a pallet from aisle 14 and bring it to bay 3. The building is 120 metres long and 60 wide. Between the robot and the pallet stand forty steel racks, a charging station, two parked forklifts and a puddle nobody has logged. The robot cannot see the pallet from where it stands. It has a map, a plan, and about eighty milliseconds in which to commit to a first move.

What happens in those eighty milliseconds is not learning, and it is not reasoning about meaning. It is search: the systematic exploration of possible sequences of moves until one is found that reaches the goal. Search is the oldest technique in artificial intelligence that still works exactly as advertised, and it is the backbone of every planner in this book, from a warehouse robot choosing a route to a robot arm choosing a path through a cluttered bench.

Five Lines That Define a Problem

Every search problem, however large, is described in the same five lines. Getting these lines right is most of the work.

The states. What is a snapshot of the situation? For the warehouse robot, a state is its cell on a grid: row and column.

The start. Where the search begins: cell (0, 0), the charging bay.

The actions. What the robot can do from a state: move one cell up, down, left or right.

The goal test. A question with a yes or no answer: are we at cell (7, 5)?

The path cost. What each move costs. One per cell is simple, but a ramp might cost two because it is slower, and a wet patch might cost three because the robot must slow down.

Notice that the description contains no algorithm at all. It is a description of the world, and once you have it, several algorithms can be pointed at it. This separation matters, because it means you can change your method later without rewriting your model of the world.

The description is also a **graph**: states are nodes, actions are edges, costs are edge weights. That is why search and data structures share so much machinery, and why the volumes on algorithms and on robot planning return to this chapter.

Rings and Dives

The simplest search, **breadth-first**, explores in rings. It looks at all states one move away, then all states two moves away, and so on. Because it finishes each ring before starting the next, the first time it reaches the goal it has found the path with the fewest moves.

Breadth-first keeps a queue — first in, the first out — of states discovered but not yet explored. That queue is called the **frontier**, and it is the only bookkeeping the algorithm needs.

Depth-first uses the same idea with one change: it keeps a stack instead of a queue, so the most recently discovered state is explored next. It dives down one branch until it hits a dead end, then backs up and tries another. It finds *a* path quickly and uses very little memory, but the path it finds can be long and strange.

The difference becomes brutal as the map grows. Suppose each state has ten possible actions and the solution is six moves deep.

- Breadth-first may need to hold about a million states in memory at the deepest ring it touches. At 100 bytes each, that is 100 MB.
- Depth-first needs roughly 60 states, a few kilobytes.

Sixteen thousand times less memory, at the cost of a path that may be three times longer. This trade-off — memory against quality — is the first genuine engineering decision in the subject, and it recurs in every robot planner you will meet.

There is a cheap way to have most of both. **Iterative deepening** runs depth-first with a limit of one move, then two, then three, until it finds the goal. It uses depth-first memory but finds the shortest path, because the first depth at

which the goal appears is its distance. It repeats work, but the repeated work is small compared with the final ring, so the total is only slightly worse than breadth-first and the memory saving is enormous.

The Guess That Saves a Shift

Search that knows nothing about direction is called blind, and blind search wastes almost all its time. On the warehouse map, breadth-first explores cells behind the robot with the same enthusiasm as cells in front of it.

Give it a **heuristic** — a guess at the distance still to go — and the picture changes. For a grid without obstacles, the number of moves between two cells if nothing were in the way is a good guess, and it is easy to compute: the difference in rows plus the difference in columns. Engineers call this the Manhattan distance, after the city grid that inspired it.

The algorithm that uses such a guess is called **A-star**, published by Peter Hart, Nils Nilsson and Bertram Raphael in 1968. It orders the frontier by a single number, f, which is the sum of g, the cost already paid to reach a state, and h, the estimate of the cost still to come. It always expands the state with the smallest estimated total.

Work through a small case. The robot is at (0, 0); the goal is (7, 5); a block of shelves occupies columns 3 and 4, rows 1 to 4.

- From the start, h is 7 + 5 = 12 moves. g is 0, so f is 12.
- Its neighbours have g = 1. The one moving right has h = 6 + 5 = 11, so f = 12. The one moving up has h = 7 + 4 = 11, also f = 12.
- Breadth-first would eventually expand about forty cells. A-star expands roughly fifteen, all of them within a narrow band between start and goal, because every cell far from that band has a large h and waits at the back of the queue.

On a realistic 200 by 200 map the difference is not a factor of three. It is a factor of thousands, and it is the difference between a planner that answers in five milliseconds and one that stalls for two seconds while the forklifts wait.

The condition that makes the guess safe

A-star is guaranteed to find the shortest path only if the heuristic never overestimates the true remaining cost. That property is called **admissibility**, and straight-line distance has it, because you cannot travel less than the straight line.

Break it and the failure is silent. Multiply every estimate by 1.2 to "encourage" the search toward the goal, and A-star will sometimes return a path that is not the shortest, while reporting success. No error appears. No warning is printed. The robot simply takes a slightly worse route, forever, and nobody knows.

The test is easy and rarely done: on a small map, run A-star and run breadth-first, and compare the path costs. If A-star is worse, the heuristic is lying somewhere. Ten minutes of checking on a toy problem is worth more than a week of tuning on the real one.

Method	Memory	Shortest path?	Needs a guess?	Typical use
Breadth-first	High	Yes	No	Small maps, few moves
Depth-first	Low	No	No	Existence checks, mazes
Iterative deepening	Low	Yes	No	Games with a time limit
A-star	Medium	Yes, if the guess is safe	Yes	Route planning, motion planning

The Price of Looking Ahead

Every search has a budget, and the budget is almost always time. The number of states grows exponentially with depth: with a branching factor of 10, depth 6 gives a million states, depth 8 gives a hundred million, and depth 10 gives ten billion. No amount of cleverness changes that curve. Cleverness only changes how much of the curve you have to touch.

This is why practical systems do three things at once. They use heuristics to avoid most of the graph. They prune branches that cannot possibly beat the best path found so far — a technique called branch and bound, and, in two-player games, alpha-beta pruning. And they stop when the clock runs out and return

the best answer so far, rather than the perfect answer too late.

That last habit is worth adopting early. In a robot, a good plan that arrives in time is worth more than a perfect plan that arrives after the collision.

A Machine That Looks Ahead

Deep Blue, which we met in the history chapter, is the most famous demonstration of search under a deadline. Its strength came from two components working together: special-purpose hardware that examined roughly two hundred million positions per second, and an evaluation function that scored a position by counting material, king safety, pawn structure and dozens of other features, tuned with the help of strong human players.

In the middle game it looked ahead only six to eight moves, sometimes more where the position was forced. Garry Kasparov could look further in the lines that mattered, because he could tell which lines mattered. The machine could not tell, so it looked at everything it could reach in the time available.

That is the honest picture of search in artificial intelligence. Machinery buys breadth. Heuristics buy direction. With neither, you have nothing; with both, you can beat a world champion inside a world with exact rules. And note what Deep Blue could not do: it could not recognise a chessboard in a photograph, explain a move to a beginner, or play any other game. It was, in the terms of the first chapter, extremely intelligent inside an extremely small box.

Problems

1. Draw a six by six grid. Mark the start at the bottom left and the goal at the top right. Blacken five cells as walls, leaving a route.

2. On three copies of the grid, trace breadth-first search, depth-first search, and A-star with the Manhattan distance. Number each cell as you expand it. Record three things for each: the path, its cost in moves, and the number of cells expanded.

3. Suppose each cell also has a cost of 1, except two cells on a ramp, which cost 3. Which of your three traces still returns the cheapest path? Explain the answer in one sentence.

4. A warehouse planner returns a path of 14 moves when the shortest is 12. What is the most likely cause, and what single test would confirm it?

5. Write the five-line description for a problem you solved today without thinking: finding a seat in a crowded room, or packing a bag. Then decide whether you searched in rings, dived, or used a guess.

CHAPTER FIVE

What the Experts Knew

Ask a hospital receptionist how they decide where to send a patient and they will give you a list. Chest pain goes to the emergency team, immediately, no discussion. Fever with a cough goes to the respiratory clinic. A sprained ankle at nine in the evening goes to minor injuries, and a sprained ankle at nine in the morning goes to the patient's own doctor. Nobody calls this list artificial intelligence. It is a set of rules, written down by people who know the place, and it works.

The first great commercial wave of artificial intelligence was built on exactly this observation. If a specialist's knowledge can be written as rules, a machine can follow the rules faster, more consistently and without forgetting the awkward case at the end of a long shift. The idea produced systems that genuinely helped, systems that cost millions, and a collapse that took a decade to recover from. All three are worth understanding, because the rule is not dead. It is running the interlock that stops a robot arm when a human enters the cage.

Two Kinds of Knowing

There is a distinction that runs through this whole book, and it is worth naming plainly.

Some knowledge is *declarative*: it can be written down as statements that are true or false. A patient with a fever above 38 °C and a cough should be tested for influenza. The door must not open while the robot is moving. A tax return filed after the deadline attracts a penalty.

Other knowledge is *procedural* or *statistical*: it lives in examples, in the way an experienced radiographer's eye moves across a scan, in the judgement that a sound is a fault and not a vibration. It is very hard to write down, and very easy to demonstrate a thousand times.

Rule-based systems are built for the first kind. Learning systems are built for the second. Most real problems contain both, which is why the argument between the two approaches has always been less interesting than the engineering that combines them.

A Very Small Language

A rule-based system has two parts: a **knowledge base**, which holds facts and rules, and an **inference engine**, which applies the rules to the facts and produces new facts.

A fact is a short statement with a name and some arguments:

```
fever(patient12). cough(patient12). age(patient12, 34).
```

A rule has a conclusion on the left, an arrow, and conditions on the right:

```
urgent(X) :- fever(X), cough(X), age(X, A), A > 60.
```

Read the arrow as “if”. If X has a fever, and X has a cough, and X is older than 60, then X is urgent. The letter X is a variable that stands for any patient. This notation is old, small, and still used, because it is compact enough for a person to read and precise enough for a machine to execute.

Now add a second rule:

```
respiratory(X) :- fever(X), cough(X).
```

Every patient who matches the urgent rule also matches this one. That overlap is not a mistake; it is the normal state of a knowledge base, and it forces a decision that no amount of code can make for you: when two rules apply, which one wins?

A defensible policy is that specific rules beat general ones, so a patient who matches the urgent rule goes to emergency even though they also match the respiratory rule. Write that policy down. A system whose behaviour depends on the order in which rules happen to sit in memory is a system nobody can explain, and you will be asked to explain it.

Running the Chain

There are two ways to make a knowledge base do work, and the difference is practical rather than philosophical.

Forward chaining starts with what is known and pushes. Take a patient with a fever, a cough, and an age of 34. The engine tests the respiratory rule: all conditions hold, so it adds the fact that the patient is respiratory. It tests the urgent rule: the age condition fails, so nothing is added. Two rules checked, one fact derived, and the answer is the respiratory clinic.

Forward chaining is data-driven, and it suits monitoring, where facts keep arriving and you want the conclusions kept up to date. An alarm system, a stock control system, a robot's safety supervisor: all of these want to be told what is true now and to work out what follows.

Backward chaining starts with a question and works back. The question is “is patient 12 urgent?” The engine looks for a rule whose conclusion is urgency. It finds the one above, and sets three sub-goals: fever, cough, age over 60. Fever is known, cough is known, the age condition fails, and the answer is no, with the exact path that produced it.

Backward chaining is goal-driven, and it suits diagnosis and configuration, where there are many rules and few questions. It does not care about anything unrelated to the question, which can save an enormous amount of work.

	Forward chaining	Backward chaining
Starts from	Known facts	The question
Style	Data-driven	Goal-driven
Good when	Facts arrive continuously	Few questions, many rules
Typical use	Monitoring, alarms, supervision	Diagnosis, configuration

Where the Money Went

The commercial story of the 1980s is worth telling in numbers, because the numbers explain the collapse better than any account of hype.

XCON, the configuration system we met earlier, grew from a few hundred rules to several thousand as the product line expanded. Each new machine needed new components, each component needed constraints on how it could be combined, and each constraint interacted with constraints written years before by engineers who had since left the company. Testing became a branch of its own. The system never became wrong; it became expensive to keep right, and that is the same thing to a business.

MYCIN, the medical adviser, illustrates a different cost. Its knowledge was excellent, and it was never adopted, because adoption depended on typing, hardware, liability and the willingness of busy clinicians to change what they did at three in the morning. The technical work was finished. The system was not.

The general name for the first problem is the **knowledge acquisition bottleneck**. Getting knowledge out of an expert's head and into rules is slow, expensive and iterative, and experts are famously bad at describing what they know. Ask a skilled driver how they hold a lane in wind and they will produce a description that is both incomplete and slightly wrong. Ask them to do it, and they will do it perfectly. This gap between what people can say and what people can do is the reason learning from examples eventually displaced rules for perception and control.

The Rules That Never Left

It would be a mistake to read this as the defeat of rule-based reasoning. Rules run the parts of the world where certainty is required, and they will keep running them.

Consider the interlock on a robot cell. When a light curtain is broken, power to the motors is cut. There is no probability in this, no training set, no confidence score. The rule is short, the behaviour is total, and it can be tested exhaustively. The same is true of aircraft checklists, railway signalling, payment authorisations above a limit, and the tax rules that compute a bill. In each case, a human must be able to read the logic, argue about it, and prove it was followed.

Rules have three properties that learning systems find hard to match: they are auditable, deterministic and testable case by case. A regulator can read a rule. A court can ask whether it was applied. A test suite can cover every branch. Those properties are worth a great deal in the places where being wrong is expensive, and no accuracy figure substitutes for them.

The modern pattern is therefore a stack rather than a choice. A rule layer handles what must never happen and what must be provable. A learned layer handles the fuzzy perception that tells the rule layer what it is looking at. A planner sits between them, deciding what to do next. When you debug a robot that behaves oddly, the fault is often at the seam between two of these layers, not inside either one.

Living With Brittleness

Rules have a characteristic failure, and it is worth learning to see it, because it appears in every system that uses thresholds.

The rule above says that a patient of 61 with a fever and a cough is urgent. What about 60 years and 11 months? The rule is silent. What about a temperature of 37.9 °C when the threshold is 38.0? Silent again. Real knowledge does not have sharp edges, and rules do. The result is a cliff in behaviour: nothing happens, nothing happens, and then everything happens at once.

Four defences are worth knowing.

Model the unknown explicitly. If a sensor fails and reports nothing, a naive engine treats “unknown” as “false” and silently concludes that the condition is not met. Give unknown its own state, and decide what the system should do when it cannot tell.

Add hysteresis, as the thermostat did, so that a value hovering at the threshold does not cause the system to chatter between two actions.

Use defaults, but mark them. “Assume the floor is dry unless the humidity sensor says otherwise” is a reasonable default. It becomes dangerous when nobody remembers that it is an assumption.

And where the boundaries are genuinely gradual, consider a system that reasons in degrees rather than in yes and no. That is the subject of fuzzy logic, which gets its own volume later in the programme. The short version: instead of asking whether the temperature is above 38 °C, ask how strongly it belongs to the category “high”, and let several rules contribute in proportion. It is a small change to the machinery and a large change to how gracefully the system behaves near the edges.

Exercises

1. A laptop will not switch on. You know that the charger is plugged in, the charger light is off, the battery was flat yesterday, the wall socket works with another device, and the cable is new. Write six facts and five rules that could lead to a diagnosis.

2. Run forward chaining by hand on your knowledge base, numbering each fact as it is derived. Then ask “is the charger faulty?” and run backward chaining. Count how many facts you had to touch in each case.

3. Add a rule that contradicts an existing one. Decide which should win, write the policy down, and explain in two sentences why the other choice would be worse.

4. Write the three rules you would put in a robot that shares a corridor with people, choosing only rules whose violation would be unacceptable. Then notice how much of the robot's behaviour those three rules leave undecided.

CHAPTER SIX

Evidence, Not Fact

A robot in a warehouse is told there is a pallet ahead. Its distance sensor reports 1.2 metres. The sensor is good, but not perfect: on a shiny floor it sometimes reads short, and in the cold room it occasionally drops a reading entirely. The robot has to decide whether to slow down, and the sensor has not given it a fact. It has given it evidence.

Almost everything a machine receives from the world is evidence. A camera reports brightness, not objects. A microphone reports pressure, not words. A thermometer reports a voltage, not a temperature. Every one of those readings is the result of physics that is partly unknown, electronics that are slightly different from the last unit, and a world that will not hold still. Learning to reason with evidence rather than with facts is the single largest step from the toys of Part One to the systems of Part Five.

The Box That Was Probably There

Start with the simplest possible statement about a sensor: it is right 90% of the time. That sentence contains two different numbers wearing one coat, and pulling them apart is most of the work.

- If something is there, the sensor says “there” 90% of the time. Engineers call this the **sensitivity**, or the true positive rate.
- If nothing is there, the sensor still says “there” 10% of the time. This is the **false positive rate**.

Now the question almost everyone forgets. How often is something actually there? In a busy aisle, perhaps one reading in ten has a real pallet in front of the robot. That number is the **base rate**, and without it the other two tell you nothing at all.

Suppose the robot takes a thousand readings. A hundred have a pallet. Nine hundred are empty aisle.

- Of the hundred real pallets, the sensor sees ninety.
- Of the nine hundred empty stretches, the sensor reports a pallet in ninety, because 10% of 900 is 90.

So when the sensor says “pallet”, there are ninety true reports and ninety false ones. The chance that a pallet is really there is one half.

A sensor described as “90% accurate” has produced a reading that is right half the time. This is not a paradox, and it is not a criticism of the sensor. It is arithmetic, and it is why robots carry several sensors instead of trusting one.

One Line of Arithmetic

The rule that turns a prior belief and a piece of evidence into an updated belief is named after Thomas Bayes, an eighteenth-century minister who never published it himself; his friend Richard Price found the manuscript and sent it to the Royal Society in 1763.

In words: the belief after the evidence is proportional to the belief before it, multiplied by how well the evidence fits.

In symbols, for a hypothesis H and evidence E:

P(H given E) = P(E given H) x P(H) / P(E)

The three ingredients have names worth knowing. **P(H)** is the prior, what you believed before. **P(E given H)** is the likelihood, how probable the evidence would be if the hypothesis were true. **P(H given E)** is the posterior, what you should believe now.

Fill in the warehouse numbers. The hypothesis is “there is a pallet”. The prior is 0.1. The likelihood is 0.9. The total probability of a “pallet” reading is the true reports plus the false ones: 0.09 + 0.09 = 0.18.

The posterior is 0.09 / 0.18, which is 0.5.

Read that again as a story rather than a calculation. The robot believed there was a pallet with probability 0.1. One reading moved that belief to 0.5. That is a large change, and it is still not enough to justify stopping a forklift.

So add a second sensor with the same characteristics, pointing at the same place from a different angle. Run the arithmetic again, using 0.5 as the new pri-

or:

Posterior = (0.9 x 0.5) / (0.9 x 0.5 + 0.1 x 0.5) = 0.45 / 0.5 = 0.9

Two readings, each half-right on its own in this situation, produce a belief of 0.9. A third takes it above 0.97. This is **sensor fusion**, and the arithmetic you have just done by hand is the core of the volume on robotic perception.

There is a condition, and it is easy to violate. The multiplication only works if the two sensors fail independently. Two cameras on the same bracket, blinded by the same low sun at the same moment, are not two pieces of evidence. They are one piece of evidence, counted twice, and the arithmetic will tell you a comforting lie.

What It Costs to Be Wrong

A robot never gets to act on certainty. It gets to act on a belief, and the right action depends on what happens if the belief is wrong.

Suppose the robot can stop, which costs two seconds of delay, or continue, which risks a collision with a forklift. Give the collision a cost of a thousand units, purely to represent that it is very bad.

If the robot continues, its expected cost is 1,000 x (1 - p), where p is its belief that the way is clear. If it stops, the cost is 2. Continuing is the better bet only when

1,000 x (1 - p) < 2

which is true only when p is greater than 0.998.

A belief of 99.8% before moving. That sounds absurd until you watch a robot in a real warehouse, where machines slow down near blind corners and at junctions, not because the software is cautious by temperament, but because somebody did this arithmetic and set the speed accordingly.

The lesson generalises beyond robots. The action threshold is not a property of the probability. It is a property of the consequences. Two systems with identical beliefs should behave differently if one can recover from a mistake and the other cannot. This is the whole of safety engineering in one paragraph, and it is why the phrase “the model was 99% confident” is not a defence after an accident.

Two Crashes and One Sensor

On 29 October 2018, a Boeing 737 MAX 8 operating as Lion Air flight 610 crashed into the Java Sea shortly after take-off, killing all 189 people on board. On 10 March 2019, another 737 MAX 8, Ethiopian Airlines flight 302, crashed six minutes after take-off, killing 157. The two accidents were similar enough that the aircraft was grounded worldwide within days.

Investigators found that a software system, designed to push the nose down when it detected a risk of stalling, was receiving data from a single angle-of-attack sensor on the side of the fuselage. That sensor was faulty. There was no second sensor cross-checking it, and the pilots had not been told that the system existed, let alone how to override it.

The first lesson is about evidence, and it is the one this chapter can compute. One sensor, however good, is a weak basis for an action that cannot be undone. A second independent source would have moved the belief from “probably a stall” to “something is wrong with the instruments”, and the disagreement itself would have been information. The absence of a second opinion was not an oversight in the mathematics. It was a decision about cost, taken by people who did not do the arithmetic in this chapter.

The second lesson belongs to a later volume, and to a later chapter of this one. A system that acts on the world must let a human see what it believes, and must give that human a way to stop it. Evidence handling and interface design are not separate subjects. They meet in the cockpit, and in the operating theatre, and in the factory.

Confidence Is Not Truth

Modern systems attach a number to every answer. A classifier says “cat, 0.97”. A language model says it is fairly sure. It is tempting to read those numbers the way you read a fuel gauge, and it is a mistake.

A number is **calibrated** if, among all the cases where the system says 0.9, it is right about nine times in ten. Many systems are not calibrated at all. They produce confident answers because the arithmetic of their output layer produces large numbers when inputs are far from anything familiar, which is exactly backwards from what a cautious engineer would want.

The test is simple and rarely run. Collect a few hundred predictions with their confidence, sort them into buckets, and check the hit rate in each bucket. If the 0.9 bucket is right 60% of the time, the system is overconfident, and every threshold you set from those numbers is wrong.

You cannot fix this by asking the model to be more careful. You fix it by measuring, by recalibrating on held-out data, and by never letting a single confidence number authorise an irreversible action.

Problems

1. A factory alarm has sensitivity 0.95 and a false positive rate of 0.05. A fault occurs in 2% of production runs. Imagine 10,000 runs and count four things: runs with a fault, true alarms, false alarms, and total alarms. Then compute the probability that there is a fault, given an alarm.

2. Repeat the calculation with a false positive rate of 0.01. Which change helps more: halving the false alarm rate, or raising sensitivity from 0.95 to 0.99? Compute both.

3. Stopping a production line costs 1,000 units. Letting a fault through costs 50,000. Using your answer to question 1, what belief should trigger a stop? Does the real alarm meet it?

4. A door sensor has sensitivity 0.92 and a false positive rate of 0.08. Doors are genuinely open in 15% of readings. Compute the probability a door is open after one positive reading, and after two independent positive readings. How many readings would you require before letting a robot pass through, and why?

5. Write down one number you have seen quoted about a system's reliability, and the three numbers you would need in order to judge it.

PART THREE

Learning and Measuring

CHAPTER SEVEN

Learning from Examples

A maintenance team has a year of records: for each week, the vibration of a pump motor, its temperature, and whether the pump failed in the following month. Nobody wrote a rule for predicting failure. There is no engineer who can tell you the exact vibration level at which a bearing starts to go, because it depends on the pump, the load, the alignment and the weather. But the records exist, and somewhere inside them is a pattern.

This is the situation that machine learning was invented for, and the shift it represents is easy to state. Instead of asking a person to write the rule, you show the machine examples and let it find the rule. What the machine finds is not wisdom. It is a compact description of the examples, and whether that description is useful depends entirely on whether the examples resemble the world the machine will meet. That is why half of this chapter is about testing, and why the next two chapters are about measurement.

Twelve Pumps

Learning begins with a table. Rows are examples, columns are **features**, and the last column is the **label** — the thing you want to predict.

Pump	Vibration (mm/s)	Temperature (°C)	Failed within a month?
1	1.2	41	No
2	1.8	43	No
3	2.1	45	No
4	2.6	47	No
5	3.1	49	No
6	3.4	52	Yes
7	3.6	51	Yes

Pump	Vibration (mm/s)	Temperature (°C)	Failed within a month?
8	4.0	55	Yes
9	4.4	58	Yes
10	4.8	60	Yes

Ten rows is a very small dataset, and it is enough to see the method. Look down the vibration column. Below about 3.2 mm/s, every pump survived. Above it, every pump failed. There is a clean gap between 3.1 and 3.4.

That gap is a gift, and the first thing to do with a gift is to notice it before reaching for anything complicated.

A Line in the Sand

The simplest model you can build from this table is a threshold:

> if vibration is above 3.25 mm/s, predict failure; otherwise predict no failure.

The number 3.25 is the midpoint of the gap, which is the safest place to put a boundary when you do not know more. The model has one parameter and can be written on the back of a receipt.

Before dismissing it, consider how much of industrial practice looks like this. A vibration limit, a temperature cut-out, a pressure relief valve: all of them are thresholds, and many of them are still doing useful work decades after they were set. The threshold is not less intelligent than a neural network. It is a different amount of assumption, and assumptions are only worth making when the data supports them.

The cost of a threshold is that it cannot express interactions. What if the pump is fine at 3.5 mm/s when it is cool, and dangerous at 3.5 mm/s when it is hot? A single threshold cannot say that. A model with two features can.

Neighbours, and the Trouble With Distance

A second simple method is **nearest neighbour**. To predict a new pump, find the most similar row in the table and copy its label.

A new pump has vibration 3.5 mm/s and temperature 50 °C. The nearest row is pump 6, at 3.4 and 52, which failed. So predict failure. If the new pump were at 2.9 and 46, the nearest row would be pump 4, at 2.6 and 47, which sur-

vived.

Nearest neighbour has an attractive honesty: it needs no training at all. It simply stores the data and compares. It also has a trap that catches almost everyone the first time.

Suppose temperature were recorded in hundredths of a degree and vibration in whole mm/s. Two pumps differing by 0.1 mm/s in vibration and 5 °C in temperature would be judged “close” in temperature terms — 500 units apart — and “far” in vibration terms, 0.1 units apart. The comparison would be decided almost entirely by the units, not by what matters.

The fix is to **scale** the features before comparing them: subtract the mean and divide by the spread, so that every feature contributes on the same footing. It is a one-line operation and it changes results enormously. When a nearest-neighbour system behaves bizarrely, the first thing to check is whether anybody scaled the columns.

Hide Four Rows First

Now the part that most introductions rush. Before choosing a threshold, before trying a second method, hide some of the data.

Pumps 3, 5, 8 and 9 go into a drawer. The model sees only the other six rows. Only after the model is fixed does the drawer open.

Using the visible rows, the gap between 3.1 and 3.4 still exists, so the threshold stays at 3.25. On the hidden four:

Held-back pump	True label	Threshold prediction
3 (2.1, 45)	No	No
5 (3.1, 49)	No	No
8 (4.0, 55)	Yes	Yes
9 (4.4, 58)	Yes	Yes

Four out of four. A perfect score, and the correct response is not celebration. It is to say plainly that four cases cannot distinguish a good model from a lucky one. The difference between four out of four and three out of four is a single pump. If the threshold had been set at 3.15 instead of 3.25, pump 5 might have flipped, and the story would be different.

Honesty about the size of the evidence is part of the skill. A student who writes “the model achieved 100% accuracy” has told you almost nothing. A student who writes “the model classified four held-out cases correctly; with four cases, the uncertainty is large, and eight more cases are needed before this result is meaningful” has told you what they know and what they do not.

Memorising the Answers

Here is where learning goes wrong in the most instructive way.

Imagine a student who has a copy of last year's examination paper with the answers. They memorise every answer. On a re-run of the same paper they score 100%. On a new paper they score 40%, because they learned the answers rather than the subject.

A model can do exactly this. Give it ten points and let it fit a curve with nine bends, and it will pass through every point. The error on the training data is zero. The curve between the points, and beyond them, wanders wildly, because the extra bends were used to accommodate noise rather than structure.

Model	Error on the ten training pumps	Error on twenty new pumps
Threshold on vibration	0 of 10	3 of 20
Curve with nine bends	0 of 10	9 of 20

The simpler model is wrong on more training cases and right on far more new ones. That trade-off is the whole content of the phrase **overfitting**, and it explains why a perfect training score is a warning rather than good news.

The remedies are not mysterious. Use fewer parameters when the data is small. Keep a held-out set and judge only on that. Add a penalty for complexity, which is called regularisation and gets its own treatment later in the programme. And above all, remember what the ten pumps can and cannot support: with ten examples and one clean gap, a threshold is about as much structure as the evidence can bear.

Exercises

1. Copy the pump table, but hide the labels of pumps 2, 5, 7 and 10. Choose a threshold using only the visible six rows, and write down how you chose it. Predict the four hidden pumps, uncover the labels, and build a confusion matrix for the four predictions.

2. Repeat with a threshold 0.5 mm/s higher and with one 0.5 mm/s lower. Which pumps change? Which kind of mistake got worse?

3. Compute the mean and spread of the vibration column and of the temperature column. Scale both columns, and recompute the nearest neighbour for a pump at 3.5 mm/s and 50 °C. Does the answer change? Explain why in one sentence.

4. Sketch, on paper, ten points that lie roughly on a rising line with visible scatter. Draw a straight line through the middle of them. Then draw a curve that passes through every point. Which one would you trust for a point at the far right of the page, and why?

CHAPTER EIGHT

The Cost of Being Wrong

Every model makes mistakes, and every project eventually has to answer a question that sounds simple and is not: how wrong is acceptable?

The question is not answered by accuracy, because accuracy treats all mistakes as equal. In most real settings they are not equal at all. A smoke alarm that wakes you at three in the morning for burnt toast is an annoyance. A smoke alarm that stays silent during a fire is a catastrophe. Any measure that scores those two errors the same has misunderstood the job.

So before a model is trained, somebody has to decide what a mistake costs. That decision is not technical, and pretending otherwise is how projects end up optimising the wrong thing with great precision.

Two Ways to Punish a Mistake

Start with a single prediction. A robot arm predicts a reach of 0.52 m; the true distance is 0.50 m. The error is 0.02 m.

If you add raw errors across many cases, positive and negative cancel. A miss of +0.3 m and a miss of -0.3 m add to zero, and the model looks perfect. So the sign has to go, and there are two common ways to remove it.

Squaring. The squared error here is 0.0004 m^2. Squaring has a second effect that matters more than the first: it punishes large misses much harder than small ones. A miss of 0.1 m costs 0.01 in squared units. A miss of 0.3 m costs 0.09 — nine times the cost for three times the error.

Averaging the squared errors across the data gives the **mean squared error**, usually written MSE. Taking its square root gives the **root mean squared error**, RMSE, which is back in the original units and therefore readable.

Absolute value. The absolute error is 0.02 m, with no sign and no squaring. Averaging gives the **mean absolute error**, MAE. Every miss counts in proportion to its size; nothing is amplified.

Take three errors: 0.02 m, 0.04 m and 0.10 m.

- Squared: 0.0004, 0.0016 and 0.01. They sum to 0.012 m². Divide by three: MSE is 0.004 m². The square root is 0.063 m.
- Absolute: 0.02, 0.04 and 0.10 sum to 0.16. Divide by three: MAE is 0.053 m.

Both are correct. They answer slightly different questions. RMSE is dominated by the worst cases; MAE describes the typical case.

The Tail Is the Story

Change one number and the difference becomes dramatic. Suppose the errors are 0.02, 0.04 and 0.30 m — perhaps a sensor glitch, or a genuine event.

- Squared: 0.0004, 0.0016 and 0.09. MSE is 0.0307 m², RMSE is 0.175 m.
- Absolute: 0.02, 0.04 and 0.30. MAE is 0.12 m.

The outlier now drives RMSE almost entirely. A model trained to minimise MSE will bend itself towards that case, because that is where the punishment is concentrated. A model trained on MAE will accept a little more error everywhere in exchange for not chasing the outlier.

Which is right? It depends on whether the outlier is noise or signal. If the 0.30 m came from a faulty sensor, MAE is the better choice, because you do not want a glitch to reshape the model. If it came from a person stepping into the robot's path, you want the model to care very much, and MSE is the better choice.

This is the sentence to carry out of the chapter: **the way you score a mistake is how you tell the model what you care about.** It is not a detail added at the end. It is a statement of values, written in arithmetic.

Of the Flagged and of the Real

Classification problems have a different shape of error, and the vocabulary is worth knowing precisely, because it is used carelessly in almost every discussion of these systems.

Take a test that looks for cracks in welds. On a batch of 100 welds, 10 are cracked. The system flags 14 as cracked. Of those 14, 8 really are cracked, and 6 are sound. It misses 2 cracked welds entirely.

	Flagged	Not flagged
Actually cracked (10)	8	2
Actually sound (90)	6	84

Accuracy is (8 + 84) / 100, or 92%. That number is true and nearly useless, because it hides the two different failures.

Precision asks: of the ones we flagged, how many were real? Here, 8 / 14 = 57%.

Recall asks: of the real ones, how many did we catch? Here, 8 / 10 = 80%.

A system with 57% precision sends six sound welds to inspection for every eight real cracks. An inspector who has to re-check six good parts for every eight bad ones will, quite reasonably, stop trusting the system within a month. And a recall of 80% means one cracked weld in five leaves the factory.

Neither number alone describes the system. Precision without recall is a system that flags almost nothing and is right when it does. Recall without precision is a system that flags everything. The pair together, with the costs attached, is a description.

Choosing the Threshold from Costs

A classifier does not usually output a yes or a no. It outputs a probability, and a threshold turns that into a decision. Moving the threshold trades one error for the other, and the trade should be made with costs, not with taste.

Suppose a false alarm costs 20 units — an unnecessary inspection — and a missed crack costs 500 units, because a cracked weld that fails in service is a disaster. The system scores 100 welds.

At the current threshold it catches 8 of 10 cracks and raises 6 false alarms:

> Expected cost = 2 misses x 500 + 6 false alarms x 20 = 1,000 + 120 = 1,120 units

Now lower the threshold so that it catches all 10 cracks but raises 30 false alarms:

> Expected cost = 0 x 500 + 30 x 20 = 600 units

The second setting is cheaper, by nearly half, even though its precision is far worse and its accuracy is lower. Anyone judging by accuracy would have

chosen the first. Anyone judging by the cost of being wrong would choose the second, and would be right.

That is the entire argument for cost-sensitive evaluation, done in four lines. It also explains a political fact about real projects: the person who defines the costs has more influence over the system's behaviour than the person who writes the code.

When You Cannot Agree on Costs

Sometimes the costs are genuinely contested. A hospital screening programme may find that a false alarm costs a patient weeks of anxiety and a follow-up procedure, while a missed case costs a life, and no committee can agree on the exchange rate.

In that situation, do not invent a number and hide it inside the model. Instead, produce the curve. Report the recall achieved at every false alarm rate, and let the clinical team choose a point on it. The technical contribution is the curve. The choice is theirs, and it should be visible, recorded and revisited.

The worst outcome is a system whose threshold was set by an engineer's intuition, deployed for years, and never examined. The second worst is a system tuned to maximise accuracy on a dataset whose class balance happens to reward doing nothing.

Problems

1. Five reach predictions for a robot arm are 1.02, 0.97, 1.10, 0.95 and 1.30 m. The true distance is 1.00 m. Compute the MSE, the RMSE and the MAE by hand.

2. Remove the 1.30 m case and compute all three again. Which measure changed most, and why?

3. A defect detector scores 100 parts: 12 defective, 9 caught, 15 flagged. Build the confusion matrix and compute accuracy, precision and recall. Write two sentences a production manager would find useful, and two they would find misleading.

4. False alarms cost 10 units; a missed defect costs 300. Compute the expected cost at a threshold that catches 9 of 12 with 15 flags, and at one that catches 12 of 12 with 40 flags. Which would you deploy, and what would

you tell the manager who preferred the other?

5. Name one decision in your own life where the two kinds of mistake are clearly unequal in cost. Write the ratio honestly, as a number, and then decide what evidence you would need before acting.

CHAPTER NINE

How Wrong Is It, Really?

There is a particular kind of meeting that anyone who has worked on these systems will recognise. A team presents a model. The accuracy is 94%. The room is pleased. Somebody asks how it was measured, and the answer is a dataset that the team has been looking at for six months.

The number is real. It is also, in a sense that will become clear, a measurement of the team rather than of the model. This chapter is about the difference, and about the habits that keep a result honest after the novelty has worn off and the system is running at three in the morning with nobody watching.

Three Sets, Not Two

The basic rule from the learning chapter is that you judge a model on data it has never seen. In practice that rule needs one more split, because development involves choices.

Divide the data into three parts.

Training data is what the model learns from. **Validation** data is what you use while deciding between models, tuning thresholds and trying features. **Test** data is opened once, at the end, to report a number you can defend.

The reason for the middle set is that every time you look at a score and adjust something in response, you leak a little information from that score into the model. Do it fifty times and the validation set has become training data by another name. Keep a set that you have never looked at, and the final number means something.

For small datasets, three-way splits waste data. A technique called **crossvalidation** helps: divide the data into, say, five parts, train five times with a different part held out each time, and average the results. Every case gets used for training four times and for testing once. It is slower and it is honest, and for the pump table in the last chapter it would have told you clearly how little ten rows

can support.

Leakage, and How It Gets In

The most common way a project fools itself is not a statistical subtlety. It is a column in the spreadsheet that contains the answer.

A maintenance dataset includes a column called “repair date”. It is empty for pumps that have not failed, and filled for pumps that have. Train on that column and the model achieves near-perfect accuracy, because it has learned to read the future. The fix is to ask, of every column: would I know this value at the moment I need to make the prediction? If not, the column is not a feature.

Leakage arrives in less obvious forms as well.

Duplicates. The same photograph appears in both training and test sets, resized or cropped. The model memorises it and scores well on a test that is not a test.

Group leakage. X-rays from the same patient appear in both sets. The model learns the patient's anatomy rather than the disease. Splitting by patient is the fix.

Time leakage. A model that predicts tomorrow's demand is trained on a random sample of days, including days after the ones it is tested on. Split by time, always, when the task is forecasting.

Preprocessing leakage. The mean and spread used to scale the features are computed over all the data, including the test set. The model then knows something about the test set before it sees it. Fit the scaling on training data only.

None of these produce an error message. All of them produce a number that is too good, and a system that disappoints in service.

The Benchmark Trap

Benchmarks have driven the field forward, and they have also taught it to cheat.

The ImageNet competition, which ran from 2010, asked systems to classify photographs into a thousand categories. In 2012 the winning deep network reached a top-five error of about 15%, against roughly 26% for the runner-up, and the result changed the direction of the field. By 2015 the best systems were below 4%, and the benchmark had, in a sense, been solved.

Something similar happened in language. Benchmarks such as GLUE measured how well systems handled a range of comprehension tasks. They were useful. They were then optimised against so hard that they stopped discriminating between good systems and lucky ones.

The mechanism is worth naming, because it is the same mechanism as overfitting, applied to research communities instead of models. When thousands of teams optimise against a fixed test set, information about that set leaks into the field. Results improve faster than the underlying capability. Then a new, harder benchmark appears, and the improvement slows abruptly, which is evidence that some of the earlier progress was aimed at the test rather than at the task.

The practical defence is the one your grandmother would recommend: keep a test set that is yours, drawn from your own world, and never publish its contents. A general benchmark tells you whether a method is promising. Only your own data tells you whether it works for you.

A Service That Drifted

In 2008, Google launched Flu Trends, which estimated the level of influenza in a region from the volume of flu-related searches. It was a genuine success. In the United States, its estimates tracked official surveillance data closely, and it reported them about two weeks earlier, because people search before they see a doctor. The paper describing it, published in *Nature* in 2009, became a standard example of what large-scale data could do.

Then it drifted. For several seasons, including 2012–13, it substantially overestimated the number of cases; some analyses put the error at more than double the true figure. Google changed its search suggestions. People's habits changed. Media coverage changed what people typed. The model had been fitted to a world that no longer existed.

There was a methodological problem underneath the commercial one. The search terms had been selected partly by checking how well they matched past flu data — including the very seasons later used to judge the model. That is a mild form of the leakage described above, and it flattered the early results. The service was discontinued in 2015.

The lesson is not that search data is useless for epidemiology. It is that a model is a hypothesis about a distribution, and distributions move. Any system that does not measure itself after deployment is, sooner or later, reporting the

past.

Confidence, Revisited

The previous chapter introduced calibration as a property of a single prediction. At the level of a whole system, it becomes a monitoring question with a simple test.

Sort the system's outputs into buckets by stated confidence. In the 0.8 bucket, check how often it was right. If it was right 0.8 of the time, that bucket is calibrated. If it was right 0.65 of the time, the system is overconfident by a wide margin.

Two things follow. First, thresholds set from those numbers are wrong, because they were chosen as if the confidence meant what it said. Second, the system's own internal sense of difficulty is broken, so it cannot tell you when to ask for help.

Calibration is fixable, and it is measured on held-out data like everything else. What is not fixable is the belief that a softmax output is a probability in the everyday sense. It is a number produced by an arithmetic operation, and it becomes a probability only after you have checked.

A Short List That Survives Contact

If you remember nothing else from this chapter, keep these five habits. They are unglamorous and they catch most of the failures that matter.

Split by time, by group and by person, not at random, whenever the real deployment will meet new times, groups or people.

Ask of every feature whether it would be available at prediction time. If not, remove it, even if it is delicious.

Fit every transformation on training data only, and apply it unchanged at run time.

Keep a private test set drawn from your own world, and open it once.

After deployment, measure again every month, on fresh labelled data, and plot the number over time. A flat line on that plot is the only evidence that a model is still working.

Exercises

1. A hospital wants a model that predicts which patients will be readmitted within a month. Write down three columns that would leak the answer, and explain in one sentence each how the leak works.

2. Design the split for that model. Do you split by patient, by admission, by hospital, by date, or by some combination? Justify each choice, and say what would go wrong if you split at random.

3. A vendor reports 97% accuracy on a public benchmark. Write the five questions you would ask before believing that this implies anything about your hospital.

4. Sketch the monthly monitoring chart for a deployed model: what is on the vertical axis, what triggers an alarm, and what would you do when it fires?

5. Think of a number you have quoted about a system's performance — from a news story, a product page or a lecture. Rewrite it so that it states the dataset, the split, the date and the uncertainty. If any of those is missing, say what is missing.

PART FOUR

The Senses

CHAPTER TEN

Pictures, Sounds, Words

Show a person a photograph of a kitchen and they see a kettle, a mug, a window, and light coming from the left. A computer receives a grid of numbers. Nothing in that grid says “kettle”. The distance from the grid to the word is the subject of vision.

Speech presents the same problem in a different costume: a microphone measures air pressure thousands of times a second, and somewhere in that measurement is a sentence. Text presents it a third time: a string of symbols whose meaning depends on the symbols around it.

Three different inputs, three different traditions of engineering, and one shared structure. All three turn raw numbers into features, features into decisions, and decisions into meaning that depends on context. This chapter is a tour of that structure, at the level of the ideas you would need before the deeper treatment in later volumes.

Nine Hundred Thousand Numbers

An ordinary photograph is a grid of **pixels**. A modest one, 640 by 480, contains 307,200 pixels. In colour, each pixel holds three numbers — red, green and blue — each between 0 and 255. The whole picture is therefore 921,600 numbers.

A full high-definition frame is 1920 by 1080, which comes to just over two million pixels and 6.2 million numbers. A camera producing 30 of those per second generates 186 million numbers every second.

These figures explain a great deal of engineering. They are why a robot's processor heats up, why video compression exists, and why the first step in almost every vision system is to throw information away. A feature is, by definition, a small summary that keeps what matters and discards the rest.

The first useful feature in most systems is an **edge**: a place where brightness changes sharply. Find the pixels where the number jumps from 60 to 180 and you have found the side of a door, the rim of a cup, the boundary between a road and a kerb. Edge detection is one of the oldest operations in the field and still the first step in industrial systems that measure a machined part to a hundredth of a millimetre.

The second step is to group edges into shapes, and the third is to name the shapes. The old way was to write rules: two vertical edges 0.2 m apart with a horizontal edge on top is a door. That works until the door is open, or the light changes, or the wall is the same colour as the door.

From Edges to Objects

The modern way is to learn the grouping from examples, and the trick that made it work is small enough to describe in a paragraph.

Instead of connecting every pixel to every other pixel, a **convolutional** network slides a small filter across the image, looking at a patch of a few pixels at a time. The filter is a tiny pattern detector: a vertical edge, a corner, a patch of texture. Early layers learn filters that respond to edges. The next layers combine those responses into corners and curves. Later layers combine those into parts of objects, and the last layers combine parts into objects.

Nobody writes down the stages. They emerge from examples, because the network is rewarded for producing the right label and free to invent whatever internal features help. That freedom is why the approach works so much better than hand-written rules, and it is also why the result is hard to explain: the features are whatever minimised the error, not whatever a person would have chosen.

It is worth keeping one number in mind, because it explains a lot of behaviour. A convolutional network's first layer might have 64 filters, each 3 by 3 pixels. That is 576 weights. Its final layer might have a million. The capacity to memorise lives in the later layers, and so does the tendency to overfit. When a vision model fails on a slightly different camera, the failure is usually in the layers that learned something too specific about the training photographs.

A Picture of Sound

Sound is a pressure wave. A microphone samples it, typically 16,000 or 44,100 times per second. One second of speech at 16 kHz is 16,000 numbers, and those numbers change completely if the same word is said louder, or by a different person, or a little faster.

The representation that solves this is the **spectrogram**. Break the sound into short slices — 25 milliseconds is typical — and for each slice record how much energy is present at each frequency. With a 10-millisecond step you get about a hundred slices per second; with 128 frequency bands per slice you get a grid of roughly 100 by 128 numbers.

That grid is a picture. Time runs left to right, frequency runs bottom to top, and brightness shows energy. Vowels produce broad horizontal bands; consonants produce short bursts of noise; a rising pitch produces a rising line.

Once speech is a picture, the tools of vision apply, and this is precisely what happened. Speech recognition improved sharply when the field began treating spectrograms with the same machinery it used for images. The problem was never really about sound. It was about patterns, and the patterns became visible once the right picture was made.

There is a cost worth noticing. A spectrogram throws away phase, which is part of how the wave is shaped, and it blurs events shorter than its slice. For recognising words that is acceptable. For locating a click to the microsecond, it is not, and a different representation is needed.

Words as Points

Text arrives as characters, and the first step is to cut it into **tokens**. A token is often a word, but not always: “robotics” might become “robot” and “ics”, because the second half appears in thousands of other words and the system can learn from that.

Each token is then mapped to an **embedding**: a list of numbers, a few hundred long, positioned so that words used in similar ways end up close together. Nobody assigns those positions by hand. They are learned from enormous quantities of text, and the geometry that results is surprisingly meaningful. The distance between “torque” and “force” is smaller than the distance between either and “banana”.

Embeddings solve one problem and create another. They give a word a position, but a single position cannot capture the fact that the same word means different things in different sentences:

> The bank raised its interest rate. We sat on the bank of the river. Bank left at the next junction.

A system that gives each word one fixed vector must fail on at least two of these. What it needs is a representation that depends on the neighbours, and the mechanism that provides it is called **attention**. Each token looks at the other tokens in the sentence, decides which ones matter for its meaning, and adjusts its representation accordingly. The paper that introduced the transformer architecture in 2017 was built on this idea, and the language systems that reached the public in 2022 are large descendants of it.

The word “attention” is doing a lot of work here, and it is worth being precise. It does not mean the system is paying attention in the human sense. It means that, for each position in the sequence, the network computes a weighted average of the other positions, with weights that depend on the content. It is arithmetic with a suggestive name, and the name has caused a decade of over-interpretation.

Three Ways These Systems Fail

Systems that see, hear and read fail in ways that are specific, and knowing them in advance is most of the defence.

They fail on the unfamiliar. A vision system trained in daylight degrades at dusk. A speech system trained on one accent struggles with another. A language model trained before an event does not know the event happened. The failure is not random; it is concentrated in whatever the training data did not contain.

They fail on adversarial input. In 2018, researchers showed that changing a few pixels — invisible to a person — could make a classifier give a confidently wrong answer. The picture still looks like a panda, and the network says “gibbon” with 99% confidence, because it is responding to a pattern that people do not perceive. In the physical world, a small sticker on a road sign has been shown to confuse some driving systems.

They fail fluently. A language model that does not know something will often produce a confident, well-formed, wrong answer rather than admitting ignorance. The form of the answer is learned from text; the content is generated by what is statistically plausible. Fluency and truth are different properties, and only one of them is measured by how good the sentence sounds.

There is a common thread. These systems model the data they were trained on, and they have no independent access to the world. When the world differs from the data, or when the input is crafted to exploit a pattern the model has learned, the output can be confidently wrong. The engineering response is always the same: test outside the training distribution, add a check that does not share the model's weaknesses, and keep a human able to see what the system is doing and stop it.

Things to Try

1. Compute the number of values in a 1280 by 720 colour image. Then compute how many arrive per second from a camera running at 25 frames per second. Compare with the capacity of an ordinary phone's storage and say what this implies for a robot recording video for an hour.

2. Take a photograph on your phone and open it in any editor that shows pixel values. Find an edge and write down the approximate values on each side. Explain, in two sentences, why an edge detector can find that boundary without knowing what the object is.

3. Record yourself saying a sentence twice: once normally, once in a whisper. Look at the waveform and, if you can, at a spectrogram. Which parts stay similar, and which change? Relate your answer to why speech systems use frequency rather than raw amplitude.

4. Write three sentences in which the word “spring” has three different meanings. Then explain what a system with one fixed vector per word would get wrong, and how attention would help.

5. Find a claim that a system “understands” images or language. Rewrite it as a testable claim about a specific dataset, a specific measure and a specific failure rate.

PART FIVE

The World

CHAPTER ELEVEN

The Chain Around the Model

A paint shop in an automotive plant installs a vision system to find scratches on door panels. The model is excellent. In testing it finds 97% of scratches longer than two millimetres, and it raises false alarms on fewer than two panels in a thousand. The team is proud of it, and rightly; the work took a year.

Six months after launch, the line manager asks for it to be switched off. The camera housing had slowly filled with paint mist. The lighting had been replaced with a slightly cooler tube. One panel in five was now being flagged, operators had learned to override the alarm without looking, and the system had become a button that people pressed to make a noise stop.

Nothing in that story is a failure of the model. The model did exactly what it was trained to do, on inputs that had drifted away from the ones it knew. Everything that failed was outside the model, in the chain that connects a mathematical object to a factory.

Six Links

A working system is a chain, and the model is one link in it.

The problem. Not “use AI to improve quality” but “find scratches longer than 2 mm on the painted surface of panel type X, at the end of the paint line, and raise an alarm within 200 ms”. The second sentence contains an object, a defect definition, a location, a deadline and an implicit benchmark: a human inspector. The first contains a wish.

The sensing. A camera on a bracket that vibrates, under a lamp that ages. The model was trained on images from a fixed rig in a laboratory. If the production camera sits two centimetres further away, every scale in the image changes and the model is looking at a different problem.

The pre-processing. Cropping, resizing, colour correction, normalisation. Every step must be identical at training time and at run time. A resize that inter-

polates differently in the two pipelines is enough to move accuracy by several points, silently.

The inference. How long does a prediction take, on the actual hardware, with the actual batch size? A model that runs in 40 ms on a workstation with a graphics card may take 300 ms on the industrial PC in the cabinet. If the requirement is 200 ms, the model is not deployable, whatever its accuracy.

The decision. The model outputs a number. Something must turn it into an action. That something is a threshold, and the threshold should come from the cost of the two kinds of mistake, as the previous chapters argued. A threshold chosen to maximise accuracy on a balanced test set is almost never the right threshold for a line where defects are one in a thousand.

The feedback. When the system is wrong, who finds out, and how does the information get back? An operator with no way to report a miss is an operator who will eventually stop caring.

Link	Typical failure	Cheap check
Problem	Requirement unmeasurable	Ask the line manager to sign the measure
Sensing	Light, dust, vibration, ageing	Photograph a reference part every hour
Pre-processing	Train/run mismatch	Log the exact input array the model receives
Inference	Too slow on target hardware	Time a thousand predictions on the real machine
Decision	Threshold set for accuracy, not cost	Recompute the threshold from the cost ratio
Feedback	No route for corrections	One button, counted weekly

A Budget Measured in Milliseconds

Latency is the part of system design that students underestimate most often, because it is invisible in a notebook and brutal in a factory.

Take the paint line. Panels pass the camera at one every two seconds. The alarm must be raised before the panel reaches the next station, 200 ms after the image is captured. Inside that budget, the pipeline has to do several things in sequence:

- capture and transfer the image: 8 ms
- crop, resize and normalise: 25 ms
- run the model: 120 ms
- compare with the threshold, log and trigger: 5 ms

That totals 158 ms, leaving 42 ms of headroom for the operating system, the network and the occasional slow frame. The headroom is not waste. It is the difference between a system that meets its deadline on a good day and one that meets it always.

Now change one number. Suppose the model took 180 ms instead of 120. The total is 218 ms, and the system fails its requirement by 18 ms. The most common response at this point is to buy a faster computer. The better response is to ask which stage can be removed: a smaller input resolution, a lighter model, or a region of interest that excludes the parts of the panel where scratches cannot occur. Engineers call this reducing the work, and it is usually cheaper and more reliable than reducing the time.

The same arithmetic appears in every robot. A control loop running at 1 kHz has one millisecond for everything it does. A drone stabilising itself has less. When you read a paper that reports excellent accuracy, look for the inference time on embedded hardware. If it is absent, the result may be true and unusable.

The Alert Nobody Read

Hospitals have been a hard testing ground for these systems, and one case is worth knowing because it involves no robot at all.

A widely used commercial system predicts which hospitalised patients are developing sepsis, a fast-moving and often fatal reaction to infection. In 2021, researchers outside the company evaluated it on patients at a single academic medical centre. The system identified about a third of the patients who went on to develop sepsis, and it raised alerts on roughly one in five of all hospitalised patients. Its measured discrimination was substantially below what the vendor's materials had suggested.

Two failures compounded. The statistical one is that a model developed and validated on one population did not transfer cleanly to another. The human one is that an alert firing on a fifth of all patients is not an alert. It is background noise, and clinicians did what people do with background noise: they stopped

hearing it.

The lesson generalises to every interface in this book. A prediction is not a decision, and a decision is not an action. Between the model and the outcome sit attention, trust, timing and workload. A system that ignores those has built an accurate component and no working product.

Who Owns It After Launch

Every deployed system needs three things that are not in the model: an owner, a review date and a retirement plan.

The **owner** is a named person, not a team, who is responsible for the system's behaviour this quarter. Without a name, maintenance becomes nobody's job, and the paint-mist problem goes unfixed for six months.

The **review date** is a calendar entry to re-measure performance on freshly labelled data. It is the only defence against drift, because drift is invisible from inside the model.

The **retirement plan** is the admission that the system will one day be wrong in a way that cannot be fixed. Products change, regulations change, cameras age, suppliers change. A system with no retirement plan is eventually switched off in a hurry by someone who has lost patience, which is exactly what happened in the paint shop.

None of these three is intellectually difficult. All three are routinely omitted, because they belong to nobody's research agenda and everybody's operational reality. The organisations that get this right treat an AI system like any other piece of production equipment: installed, maintained, inspected, and eventually replaced.

Exercises

1. Write the one-page framing for a system that would inspect something you can see in your own environment — a wall, a page, a piece of fruit. Include the object, the defect, the sensor, the time limit, the error budget and the person who signs off.

2. Draw the six links for your system and write the biggest failure risk for each. Then write the cheapest check that would catch that failure.

3. Build a latency budget for a camera at 25 frames per second with a 400 ms alarm deadline. Allocate times to capture, pre-processing, inference and decision, and show your headroom.

4. In the sepsis case, the model identified about a third of cases and alerted on a fifth of patients. Using the definitions from the previous chapters, compute precision and recall as far as the numbers allow, state what is missing, and explain why clinicians ignored the alerts.

5. Think of a system in your school, workplace or home that was installed and then forgotten. Identify the drift that made it useless, and the review date that would have caught it.

CHAPTER TWELVE

What Machines Still Cannot Do

In March 2018, a self-driving test vehicle operated by Uber struck and killed a woman walking a bicycle across a road in Tempe, Arizona. The vehicle was travelling at about 70 km/h in a 65 km/h zone. Its sensors detected the pedestrian several seconds before impact. The software classified the object repeatedly, changing its mind between vehicle, bicycle and other, and its predictions of the object's path moved back and forth. The system's design did not treat “an unknown object in the road” as a reason to slow down. The safety driver was looking away from the road, and the vehicle braked less than a second before impact.

The technical facts of that night are the clearest illustration available of what these systems can and cannot do. The perception worked: something was there, and it was seen. The classification was uncertain, and the system had no policy for acting on uncertainty about *what* a thing is, when the important fact was *where* it is. A human driver who could not identify an object in the road would slow down. The machine had not been taught that hesitation is a form of intelligence.

This chapter is about the specific limits that produce failures like that one, and about the risks that follow when such systems are deployed at scale.

Pattern Is Not Cause

A model trained on history learns associations. Ice-cream sales rise with drowning deaths; both rise with summer. The association is real, and the causal story is missing a variable.

Associations are enough when you only need to predict. They are not enough when you need to act, because actions change the world in ways the data has never seen. A hospital that learns “patients with this test ordered have worse outcomes” and then orders the test more often will change the popula-

tion, and the model will be measuring its own effect.

The formal tools for this belong to a later volume, and the instinct belongs here. Before acting on a model's recommendation, ask what would happen if you intervened. If you cannot answer, you have a prediction, not a plan. The cheapest version of the question is the one a good manager asks: “and then what?”

The World Moves Underneath

Every model is trained on a sample of the world and deployed into a world that keeps changing. The gap has a name, distribution shift, and three common forms.

Covariate shift. The inputs change but the relationship does not. A camera's colour balance drifts as the lamp ages; the mapping from image to defect is unchanged, but the images are not.

Label shift. The proportions change. A fraud model trained when fraud was one transaction in a thousand meets a season when it is one in fifty.

Concept drift. The relationship itself changes. What counted as a good credit risk in 2005 was not a good credit risk in 2009, because the economy had changed and the model had not.

The Tempe crash had an element of all three. Night, unusual lighting, a pedestrian pushing a bicycle rather than walking, and a location with a history of jaywalking. None of these were exotic. They were simply absent, or rare, in the data the system had been tuned on.

The engineering response is not to demand that the world stop moving. It is to measure the inputs as well as the outputs, to keep a labelled sample from each month, and to design the system so that unusual inputs provoke caution rather than confidence. That last one is a design decision, and it is the one that was missing in Tempe.

Common Sense and Its Absence

Ask a language model for a recipe and it will produce something plausible, well organised and grammatically flawless. Ask it to explain how to cool a sauce quickly and it may suggest putting the pan in the refrigerator, which would warm the fridge, cool the sauce slowly, and possibly warp the pan.

The model has read an enormous quantity of text about cooking. What it does not have is a model of heat, or of what a pan weighs, or of why the fridge door has a shelf that will not take a heavy object. Those facts are rarely written down, because everybody knows them. Text contains what people bothered to say, and people do not bother to say the obvious.

This is the reason a system can be fluent about a subject and wrong about its physics. Fluency is learned from the surface of language; physical intuition is learned from doing things in a world with consequences. Until a system has either, its confident prose should be read as a proposal, not a report.

The practical consequence for engineering is to keep the physical checks separate. A robot that plans a path through a wall should be stopped by a collision checker, not by a better language model. A system that suggests a temperature should have a rule that no temperature above the boiling point of the fluid is acceptable. Rules catch what learning cannot see.

The Price of the Answer

There is a cost side to all of this that rarely appears in technical papers and always appears in an electricity bill.

Training a large model consumes a great deal of energy. Estimates for the largest models run to hundreds of megawatt-hours for a single training run, before any use at all. Inference — answering a query — costs much less per query, but is paid billions of times. A system that is 2% more accurate and twice as expensive may be a worse product, and a system that cannot run on the device in the customer's hand may be no product at all.

There is a human cost as well. Supervised learning needs labels, and labels are made by people. Some of that work is skilled. Some of it is poorly paid, repetitive, and psychologically damaging: the people who review harmful content so that a model can learn to filter it are exposed to the material that the filter exists to hide.

These costs are not arguments against the technology. They are arguments for counting them. A design that ignores them will be optimised for the metric that was counted and will quietly export the rest.

Who Answers for It

In 2018, a large technology company abandoned an internal recruiting tool that had been trained on a decade of hiring decisions. The system had learned to favour candidates whose applications resembled those of people who had previously been hired, and because the industry had hired mostly men for technical roles, the tool penalised résumés that contained the word “women's” — as in “women's chess club”. The company said the tool was never used to make final decisions. It was still a system that had learned the past and would have reproduced it with the authority of arithmetic.

Bias of this kind is not a bug in the mathematics. It is the mathematics doing exactly what it was asked: find the pattern in the data. If the data encode a history of exclusion, the pattern is exclusion.

The response has to be structural. Measure outcomes by group, not just in aggregate. Ask whether the label you are predicting is the thing you actually want. Keep a human decision-maker who is accountable for the result. And make the system contestable: a person affected by a decision should be able to ask how it was made and get an answer that is not a shrug.

Accountability is the thread that ties this volume's technical chapters to the ethical ones that follow. A system that decides something about a person is a system that somebody must be able to explain. If nobody can, the system is not ready, however good its score.

Problems

1. For each of the three forms of distribution shift, invent an example from a system you have used, and say what measurement would reveal it first.

2. The Tempe system changed its classification of the pedestrian several times and had no policy for acting on that uncertainty. Write the policy you would have specified, in three sentences, and say what it would have cost in normal operation.

3. Ask a language model, or a person pretending to be one, for instructions that involve a physical process you understand well — cooking, plumbing, lifting. Find one step that is plausible in prose and wrong in the world. Explain why the training data would not contain the correction.

4. Estimate the energy cost of your own use of an online service for a week, using published figures where you can find them, and stating your assumptions. Then decide whether the service is worth it to you.

5. Write one sentence describing a decision you would not delegate to a machine, and one sentence explaining why. Keep both. In four years, read them again.

CHAPTER THIRTEEN

Where This Leaves You

The argument of this volume can be put in a page.

Intelligence, as engineers use the word, is behaviour: sensing, deciding and acting to reach a goal in situations not seen before. The name artificial intelligence attaches itself to whatever is not yet solved, which is why it tells you so little. What tells you a great deal is the shape of the machine: what it senses, what it can choose, how it is measured, and how it is tested.

Machines decide in three ways, and they are older than the name. They search, when the space of possibilities can be described and explored. They follow rules, when the knowledge can be written down and must be auditable. They weigh evidence, when the world refuses to be certain. Almost everything built today uses all three, glued together by software and judged by a number that somebody chose.

Machines learn by fitting a description to examples, and that description is only as good as the examples and the testing. A model that scores perfectly on its training data has usually learned the wrong thing, and a model that scores well on a public benchmark has usually learned the benchmark. The honest questions are always the same: what was held back, how was the split made, what does a mistake cost, and what happens after the world changes.

And machines meet the world through a chain: a problem framed, a sensor placed, a pipeline matched, a threshold chosen, a person able to override, an owner with a calendar. The model is one link. Most failures happen at the others.

What Comes Next

The programme continues for forty-seven more volumes, in the order a student should meet them.

The first year completes the foundations: the physical body of a robot and its parts; machine learning done properly; vision from pixels to objects; language from tokens to meaning; and control, which is what keeps a machine on target while the world pushes it off.

The second year adds the tools and the systems: data structures and algorithms, signals, embedded computing where memory and time are scarce, the study of how people actually use robots, neural networks from a single unit to a deep model, and the graphics that make simulation possible. Then the internet of things, robotic perception, ethics, autonomy, reinforcement learning and computer architecture.

The third year is about movement, adaptation and other minds: mobile robots, AI for robots, fuzzy logic, deep learning, pattern recognition and kinematics; then virtual reality, cyber-physical systems, swarms, robot software, cognition and biological inspiration; then industrial vision, intelligent control, teams of robots, localisation and mapping, planning, and social robots.

The fourth year is practice at the edge: evolutionary robotics, robot ethics, robot learning, AI ethics, manipulation and perception; and finally navigation, path planning, simulation, robot vision, design, and a capstone project in which you build something of your own and defend it.

Each volume is a course of about twenty thousand words. Each assumes the one before it, and each is written so that a reader who arrives late can still follow.

How to Read What Comes Next

Three habits will carry you through the programme better than any amount of talent.

Read each chapter three times. The first reading, quickly, for the shape of the argument. The second, slowly, with a pen, doing the worked examples yourself with the numbers covered. The third, a week later, reading only the headings and the closing pages. The third reading is the one that moves knowledge from the week into the year.

Do the arithmetic on paper. Every formula in this programme is small enough to compute by hand at least once. A person who has computed a Bayesian update, a confusion matrix and a latency budget with a pencil owns those ideas in a way that a person who has watched a video does not.

Build something small and finish it. Not a robot that does everything, but a robot that does one thing reliably, with a measurement attached. The discipline of finishing teaches more than the ambition of starting, and it is the difference between a student who has read about systems and an engineer who has shipped one.

A Word About the Word

The name on the cover of this book will look different in ten years. It may be replaced by something with more syllables and less meaning. The systems will be faster, cheaper and more capable, and the boundary of what counts as intelligent will have moved again, as it has moved with every generation since the 1950s.

The questions underneath the name will not have moved. What does this system sense? What can it choose? How do we know it works, on data it has not seen, in conditions we did not design for? Who is accountable when it is wrong, and what does being wrong cost, and to whom?

Those questions are older than the computer and will outlast the current fashion. If you can answer them about any machine you meet, you will be useful in a room full of people who cannot. That is the whole ambition of this volume, and it is enough to begin.

Glossary

Accuracy — the fraction of all predictions that are correct. Useful only alongside a baseline and a breakdown of mistakes.

Actuator — the part of a machine that changes the world: a motor, a wheel, a heater, a display.

Admissible heuristic — a guess at the cost still to go that never overestimates the truth. Required if A-star is to return the shortest path.

Agent — anything that senses its environment and acts on it.

AI effect — the habit of calling a solved problem "not really artificial intelligence" once it works.

A-star — a search that orders possibilities by the cost already paid plus the estimated cost still to come.

Base rate — how common a condition is in the population, before any evidence is considered.

Baseline — the score a deliberately simple method achieves. The bar that intelligence must clear.

Bias — a systematic error that disadvantages a group or a situation; also, in statistics, the difference between an estimate and the truth.

Breadth-first search — exploring in rings, nearest first; finds the shortest path and uses a great deal of memory.

Calibration — the agreement between a stated confidence and the observed frequency of being right.

Classification — predicting which category an example belongs to.

Concept drift — a change in the relationship between inputs and outputs, so that a model that was right becomes wrong.

Confusion matrix — a table of predictions against true labels, showing the two kinds of mistake separately.

Convolution — a small filter slid across an image or signal, producing a map of where its pattern occurs.

Cross-validation — training and testing several times on different splits, so that every case is used for both.

Data drift — live data slowly diverging from the data a model was trained on.

Depth-first search — following one branch to its end before trying another; low memory, no guarantee of a short path.

Distribution shift — any difference between the data a model was trained on and the data it meets in use.

Embedding — a list of numbers representing a word or token, positioned so that similar meanings are close together.

Error budget — the mistakes a system is allowed to make, specified by type.

Expert system — a program holding a specialist's rules, used in medicine, configuration and industry from the 1970s.

Feature — a measurable input used by a model.

Forward chaining — deriving conclusions from known facts by applying rules until nothing new appears.

Generalisation — doing well on cases the model has never seen.

Heuristic — an estimate or rule of thumb that guides a search without guaranteeing an answer.

Inference — in logic, deriving new facts from known ones; in machine learning, using a trained model to produce an output.

Knowledge base — the store of facts and rules in a rule-based system.

Label — the answer a supervised model is trained to predict.

Leakage — information about the test set, or about the future, accidentally present in the training data.

Loss — a single number scoring how wrong a set of predictions is. Smaller is better.

Mean absolute error — the average size of the errors, ignoring sign.

Mean squared error — the average of the squared errors; punishes large misses heavily.

Narrow intelligence — a system good at one task and helpless outside it.

Nearest neighbour — predicting by copying the label of the most similar known example.

Overfitting — fitting the training examples so closely that new examples fail.

Percept — one reading of the environment.

Pixel — one small square of an image, holding a brightness or colour value.

Policy — an agent's rule for choosing an action from what it knows.

Precision — of the cases a system flagged, the fraction that were correct.

Prior — a belief held before new evidence is considered.

Recall — of the real cases, the fraction the system found.

Regularisation — any penalty on model complexity intended to improve generalisation.

Reinforcement learning — learning a policy by trial, error and reward.

Sensitivity — of the cases that are really positive, the fraction a test detects.

Sensor fusion — combining several sources of evidence into one belief.

Spectrogram — a picture of sound, showing energy at each frequency over time.

Supervised learning — learning from labelled examples.

Threshold — a cut-off that turns a number into a decision.

Token — a small unit of text, usually a word or part of a word.

Training set — the examples a model learns from.

Transformer — a network architecture built on attention, introduced in 2017 and now dominant in language processing.

Sources and Further Reading

This volume is an introduction, and introductions should point outward. The works below are the ones a reader can go to next without being misled, and the ones from which the factual claims in this volume are drawn.

The founding papers

Turing, A. M. (1936). “On Computable Numbers, with an Application to the Entscheidungsproblem.” *Proceedings of the London Mathematical Society*. The paper that gave “computable” a precise meaning.

Turing, A. M. (1950). “Computing Machinery and Intelligence.” *Mind*. The paper that replaced a philosophical question with a test.

McCarthy, J., Minsky, M., Rochester, N., and Shannon, C. (1955). “A Proposal for the Dartmouth Summer Research Project on Artificial Intelligence.” The document that named the field.

General introductions

Russell, S., and Norvig, P. *Artificial Intelligence: A Modern Approach*. The standard reference, now in its fourth edition. Use it for depth after each chapter here, particularly for search and probability.

Wooldridge, M. *An Introduction to Multiagent Systems*. The clearest treatment of agents and environments.

Mitchell, T. *Machine Learning*. Older than most readers, and still the best account of what learning from data actually means.

Learning and neural networks

Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. Free online. The early chapters are readable after this volume.

Sutton, R., and Barto, A. *Reinforcement Learning: An Introduction*. Free online. Chapters 1 to 3 pair well with later volumes in the programme.

Rumelhart, D., Hinton, G., and Williams, R. (1986). “Learning representations by back-propagating errors.” *Nature*. The paper that made multi-layer networks trainable.

Vaswani, A., et al. (2017). “Attention Is All You Need.” The transformer paper.

Perception

Szeliski, R. *Computer Vision: Algorithms and Applications*. Free online, and comprehensive.

Ribeiro, M., Singh, S., and Guestrin, C. (2016). “Why Should I Trust You? Explaining the Predictions of Any Classifier.” The paper containing the husky-and-snow analysis discussed in the first chapter.

Systems, failure and ethics

Wong, A., et al. (2021). “External Validation of a Widely Implemented Proprietary Sepsis Prediction Model in Hospitalized Patients.” *JAMA Internal Medicine*. The evaluation discussed in the chapter on systems.

National Transportation Safety Board. Reports on the Tempe, Arizona pedestrian fatality (2018) and on the Boeing 737 MAX accidents. Dry, precise, and the best available source of real failure analysis.

Ginsberg, J., et al. (2009). “Detecting influenza epidemics using search engine query data.” *Nature*. The paper behind Flu Trends, read alongside the later critiques.

O'Neil, C. *Weapons of Math Destruction*. A journalist's account of algorithmic harm, useful for its cases rather than its theory.

Noble, S. U. *Algorithms of Oppression*. How search systems encode and reproduce bias.

Mathematics you will need

For readers who want the mathematics behind later volumes: any standard introduction to linear algebra, and any introduction to probability. The specific tools used in robotics — matrices for rotation, Bayes' rule for estimation, gradients for learning — are introduced from scratch in the volumes that use them.

Numbers in this volume are either derived in the text or drawn from the published sources above. Where a figure is an illustrative example rather than a measurement, the text says so.